

Универсальное пособие * полный исходный код

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 04.09.2026, 21:20:57
Файлов исходного кода: 61
Суммарный объем кода: 855.6 KB
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref-sum]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planar-coloring]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler-cycle]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

.....

22. src/data/tickets/ticket21_24.ts

23. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

24. src/components/ArticulationPointsViz.tsx

25. src/components/BellmanFordViz.tsx

26. src/components/BfsViz.tsx

27. src/components/ChapterImage.tsx

28. src/components/DfsBridgesSimulator.tsx

29. src/components/DijkstraViz.tsx

30. src/components/DPVisualizer.tsx

31. src/components/EulerSimulator.tsx

32. src/components/ExpressQuiz.tsx

33. src/components/FloydViz.tsx

34. src/components/GraphTraversalViz.tsx

35. src/components/HeapViz.tsx

36. src/components/JohnsonViz.tsx

37. src/components/KosarajuViz.tsx

38. src/components/KruskalSimulator.tsx

39. src/components/MnemonicCards.tsx

40. src/components/Navbar.tsx

41. src/components/PlanarityDemo.tsx

42. src/components/QueueViz.tsx

43. src/components/SalmonAutomatonWidget.tsx

44. src/components/SegmentTreeVisualizer.tsx

45. src/components/Sidebar.tsx

46. src/components/Simulator.tsx

47. src/components/SplayTreeViz.tsx

48. src/components/StackViz.tsx

49. src/components/StringAlgorithmsViz.tsx

50. src/components/TopologicalSortViz.tsx

51. src/components/WaterfallAnimationWidget.tsx

Визуализаторы (вложенные)

52. src/components/visualizers/SparseTableViz.tsx

HTML-разметка (layout)

53. src/layout/footer.html

Универсальное пособие • полный исходный код

ФАЙЛ 2/61: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2

Gemini App Canvas Instructions

Ты – AI-ассистент, работающий в Gemini App Canvas (или а
При запросе пользователя создать или сгенерировать новую
строго придерживаться готовых шаблонов разметки, :

Правила для малых страниц (Билетов / Глав)

1. ****Единый стиль:**** Всегда используй Tailwind CSS и HTML
2. ****Структура JSON/TS:**** Каждая новая малая страница д
 {`', `content: ...`) для вставки в `src/data/content.
 `
3. ****Строгая структура контента:****
 - Заголовок с цветным бейджем (например, `bg-indigo-100`)
 - Определение/правило в центре (`border-blue-500` или `border-2px solid blue`)
 - Обязательная сетка аналогий (2 колонки: неформальный и формальный)
 - "Душные" детали (код, формулы, сложные доказательства)
4. ****Не ломай верстку:**** Не придумывай свои CSS-классы и HTML-теги в файлах `src/data/content.ts`. Не используй чистый markdown.

ФАЙЛ 3/61: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 5

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1" />
    <title> Дискретка для тупых – Интерактивный гид по
```

```
.....  
  "export:png": "node scripts/export/render-pages.mjs",  
  "export:pdf": "node scripts/export/build-pdf.mjs",  
  "export": "npm run export:txt && npm run export:png",  
},  
"dependencies": {  
  "clsx": "2.1.1",  
  "lucide-react": "^1.21.0",  
  "motion": "^12.40.0",  
  "pdfkit": "^0.20.1",  
  "react": "19.2.6",  
  "react-dom": "19.2.6",  
  "tailwind-merge": "3.4.0",  
},  
"devDependencies": {  
  "@tailwindcss/vite": "4.1.17",  
  "@types/node": "22.19.17",  
  "@types/pdfkit": "^0.17.6",  
  "@types/react": "19.2.7",  
  "@types/react-dom": "19.2.3",  
  "@vitejs/plugin-react": "5.1.1",  
  "esbuild": "^0.28.2",  
  "tailwindcss": "4.1.17",  
  "typescript": "5.9.3",  
  "vite": "7.3.2",  
  "vite-plugin-singlefile": "2.3.0",  
},  
"description": "Интерактивное пособие по алгоритмам и  
}
```

.....
ФАЙЛ 6/61: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 101 | РАЗМЕР: 1000

.....
algv0 – Универсальное пособие по алгоритмам и структурам

Интерактивная web-читалка (React + Vite + Tailwind) с

Универсальное пособие • полный исходный код

Приложение отдаёт эти файлы по ссылкам `/export/full_co`

Аудит контента

```
```bash
```

```
node scripts/audit-coverage.mjs # какие темы без
```

```
node scripts/audit-coverage.mjs --md # markdown-табли
```

```
```
```

Актуальный отчёт: `[`TOPICS_AUDIT.md`](./TOPICS_AUDIT.md)`

Структура

```
...
```

```
src/
```

```
  App.tsx          — оболочка, привязка глав к визу
```

```
  components/      — интерактивные симуляторы (Дейк
```

```
  data/            — контент пособия {главы/билеты :
```

```
scripts/export/    — пайплайн код → txt → png → pdf
```

```
public/export/     — сгенерированные TXT и PDF (обн
```

```
.github/workflows/ — автоматизация
```

```
```
```

```
...
```

## # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструмен  
азуемые теги и классы.

## ## Заголовки

\* Заголовки секций: Используют стандартный тег `<h2>` и

\* Подзаголовки: Используют тег `<h3>`.

## ## Текст и параграфы

\* Обычный текст содержится в тегах `<p>`.

\* Блоки "Шиза" (Аналогии) и другая текстовая инфографика

Универсальное пособие • полный исходный код

```
.....
> * **2D** — к главе привязан интерактивный визуализатор
>
> Всего глав в пособии: **25**.
```

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

```
| Глава | id | Что делать |
| --- | --- | --- |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8
```

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет — и

```
| Билет | Тема | Статус |
| --- | --- | --- |
| **1** | Дерево отрезков (Segment Tree) | Главы нет. К
ит вхолостую. |
| **7** | Планарность и раскраска графов | Номерной гла
огии**. |
| **11** | Мосты в графах | Номерной главы нет; есть бе
| **13** | Эйлеровы пути и циклы | Номерной главы нет; с
```

Дополнительно — «осиротевшие» визуализаторы без глав (к

```
* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Дер
* `dynamic-programming` → `DPVisualizer.tsx` — **Динами
```

## Универсальное пособие • полный исходный код

12	12. Графы. Точки сочленения	1	-	-
14	14. Графы. Кратчайший путь. Дейкстра	3	-	-
15	15. Графы. Кратчайший путь. Форд–Беллман	1	-	-
16	16. Графы. Кратчайший путь. Флойд	-	-	-
17	17. Графы. Алгоритм Джонсона	-	-	-
18	18. Графы. Остовное дерево. Краскал	1	-	-
19	19. Графы. Остовное дерево. Прима	1	-	-
20	20. Графы. Остовное дерево. Боруэка	1	-	-
21	21. Префикс-функция (π), КМП	4	-	-
22	22. Z-функция	4	-	-
23	23. Алгоритм Ахо–Корасик	2	-	-
24	24. Классы сложности, сведение задач	4	-	-
-	Обзор: Кратчайшие пути и Графы	-	-	-
-	Алгоритмы на карте	-	-	-
-	Эйлер: Путь ≠ Цикл	-	-	-
-	Мосты в графах: код + визуализация	1	-	-

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» — минимум `initial`
4. **\*\*Решить судьбу `alg-map`\*\*** — раскрыть или удалить.

ФАЙЛ 8/61: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
```

```
.....
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

---

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

---

.....  
ФАЙЛ 10/61: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 134 | РАЗМЕР: 6.2 K

---



```
.....
const activeChapter = chapters.find(c => c.id === act
```

```
return {
```

```
 <div className="min-h-screen bg-slate-950 text-slate
```

```
 <Navbar activeTab={activeTab} setActiveTab={setAc
```

```
 {activeTab === 'guide' ? (
```

```
 <div className="flex flex-col lg:flex-row max-w
```

```
 {/* Sidebar */}
```

```
 <div className="lg:w-80 flex-shrink-0 w-full
```

```
 100vh-4rem)] overflow-y-auto">
```

```
 <Sidebar selectedChapterId={activeChapterId
```

```
 </div>
```

```
 {/* Main content */}
```

```
 <main id="main-content" className="flex-1 p-4
```

```
 <div className="prose prose-invert max-w-n
```

```
 dangerouslySetInnerHTML={{ __html: ac
```

```
 {/* Specialized interactives */}
```

```
 <div className="mt-8">
```

```
 {activeChapter.id === 'euler-path-vs-cyc
```

```
 {activeChapter.id === 'bridges-code' && .
```

```
 {activeChapter.id === 'graph-articulation
```

```
 {activeChapter.id === 'planarity-euler-f
```

```
 {activeChapter.id === 'graph-dfs-bfs' &&
```

```
 {activeChapter.id === 'stack-dfs' && <Sta
```

```
 {activeChapter.id === 'queue-bfs' && (
```

```
 <div className="space-y-12">
```

```
 <QueueViz />
```

```
 <BfsViz />
```

```
 </div>
```

```
))
```

```
 {activeChapter.id === 'heap-beam-search'
```

```
 {activeChapter.id === 'intro' && <Mnemon
```

```
.....
 {activeChapter.id === 'dynamic-programming' && <DynamicProgramming />
 {activeChapter.id === 'splay-tree' && <SplayTree />
 {activeChapter.id === 'scc-kosaraju' && <SCCKosaraju />
 {activeChapter.id === 'top-sort' && <TopologicalSort />
 </div>

 </main>
 </div>
) : (
 <main className="p-4 md:p-8 lg:p-12 max-w-5xl mx-auto">
 <Simulator />
 </main>
)}

 {/* Footer */}
 <footer className="p-8 text-center text-slate-600">
 © 2026 Universal Educational Guide. Интерактивное учебное пособие.
 </footer>
</div>
);
}
```

.....  
ФАЙЛ 11/61: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 50 | РАЗМЕР: 950 В

```
.....
@import "tailwindcss";

@layer utilities {
 .neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
 }
 .neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
 }
 .neon-glow-rose {

```

Универсальное пособие • полный исходный код

```
.....
}
```

```
.....
ФАЙЛ 12/61: src/main.tsx
```

```
КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В
.....
```

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

```
.....
ФАЙЛ 13/61: src/types.ts
```

```
КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В
.....
```

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
}
```

```
.....
ФАЙЛ 14/61: src/vite-env.d.ts
```

```
КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В
```

```
.....
 description: "Разреженная таблица для идемпотентных
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ух отрезков: $\min(ST[L][k], ST[R - 2^k + 1][k])$
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4">
 зок длины 7. Ты берешь две заготовленные заготовки
 ь отрезок длины 7 (перекрывая друг друга по 1 см)
 ничего складывать, просто пересекаем куски.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ужно покрыть отрезок, мы берем две самые большие
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
```

```
.....
 <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
 Аналогия 1: Удар катаной (Sp
 </div>
 <p class="text-slate-300 text-sm mb
по значению X. Все, кто меньше X улетают в.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Аналогия 2: Слияние капель (M
 </div>
 <p class="text-slate-300 text-sm mb
вого). Мы просто смотрим на корни: у кого Y
как потомок.</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
pair<Node*, Node*> split(Node* t, int x) {
 if (!t) return {nullptr, nullptr};
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}</pre>
 </div>
 </details>
```

```

.....
 </div>
 <p class="text-slate-300 text-sm mb-1">
и становится кривым как бамбук. Но как тол
 </p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">
 Код (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded"
// Zig, Zag
function rotate(x) {
 let p = x.parent;
 if (p.left === x) { // Right rotate
 p.left = x.right;
 x.right = p;
 } else { // Left rotate
 p.right = x.left;
 x.left = p;
 }
}
}</pre>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за O(1)",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">

```

```

</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ый граф можно раскрасить 4 цветами.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 i
 rder border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </section>,
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

.....

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
нвертированному в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением
 </div>
 <p class="text-slate-300 text-sm mb-4">
 БЫХ направлениях по односторонним улицам. Как это отразится
 у на карте.</p>
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает граф",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

```



```

.....
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
тей > 1).</p>
</div>
<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 b
rder border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb">
правильный роутер - и два сегмента офиса б
 </div>
 </div>
</div>
</div>
</section>',
},
{
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 </div>
 </div>
 <div class="grid grid-cols-1 gap-6">

```

```
.....
 Аналогия: Копаем туннель под
 </div>
 <p class="text-slate-300 text-sm mb-1">
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площадь
 </div>
 </div>
</div>
</div>
</section>`,
},
{
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb-1">
```

```

.....
 </div>
 </div>
</div>
</section>`,
},
{
 id: "mst-boruvka",
 title: "20. Графы. Основное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Основные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-4">
 изкому племени для объединения. К вечеру пле
 ства шлют гонцов к ближайшему чужому царству.
 </div>
 </div>
 </div>
 </div>
 </div>
</div>
</div>

```

Код (Скрыто)

```

</summary>
<div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 щегося с i.</p>
 </div>
 </div>
 <div class="grid grid-cols-1 gap-6">

```

```

.....

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 i
rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">
пример сортировка чисел). Класс NP –
енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 i
rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-4">
ь отличный переводчик на английский (Сведени
аче B, то B – это NP-Полная (сосредо
 </div>
 </div>
 </div>
</div>
</section>`,
 },
};

```

```
.....
 <p><strong class="text-white">Цикл
 </div>
</div>
</div>

<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла)

* <span class="text-green-400 font-
шел-вошёл-вышел).

* <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border-2 border-
 <div class="absolute -top-3 left-4 bg-slate-800 border-2 border-
order-green-500 shadow-md">
 Путь: Нормальная прогулка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная)
 талый пришёл в бар (вторая нечётная)
 Домой вернуться не смог, потому что
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2 border-
 <div class="absolute -top-3 left-4 bg-slate-900 border-2 border-
der-rose-500 shadow-md bg-slate-950">
 Цикл: Гамильтонов синдром (болезнь)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Гамильтон – это болезнь.
 ng> ровно раз и вернуться домой.
 Ты обходишь все бары (вершины) ровно раз.
 Главное – зайти и выйти, не заходя дважды.
 Никто не знает, как решать быстро.
 </p>
 </div>
</div>
```

```
<div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border"
```

```
 <h3 class="text-xl font-bold text-emerald-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-emerald-300 italic">
```

```
 <div class="text-left font-mono text-sm">
```

```
 <p><strong class="text-white">tin{v
```

```
 <p><strong class="text-white">low{v
```

```
 <div class="p-3 bg-slate-950 rounded">
```

```

```

```
 <p class="text-xl font-bold tex
```

```
 <p class="text-xs text-slate-400
```

```
 </div>
```

```
 </div>
```

```
 </div>
```

```
<p class="text-slate-300 text-sm">
```

```
 Если из сына и и всех его потомков <str
```

```
 Единственная ниточка. Порвётся – граф р
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap"
```

```
 <div class="bg-slate-800 p-6 rounded-lg border"
```

```
 <div class="absolute -top-3 left-4 bg-s"
```

```
 border-emerald-500 shadow-md">
```

```
 Мост: Единственная веревка
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 Представь, что ты альпинист. Ты спу
```

```
 Из пещеры и нет других выходов – то
```

```
 Если верёвка (ребро v-u) оборвётся -
```

```
 Если бы из пещеры и был чёрный ход
```

```
 </p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border"
```

```

for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
}
}
}

```

```

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
 tin.assign(n, -1);
 low.assign(n, -1);

 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
}

```

</div>

<p class="text-xs text-slate-400">Занял

</div>

</details>

</div>

</section>`,

},

{



```
.....
-white">10 > 9</code> – БАХ, непланарен!
 Все 5 вершин соединены со всеми. На
 Теорема Куратовского: K5 – эталон не
 Если внутри графа спрятан K5 – забуду,
```

</p>

</div>

```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-rose-500 shadow-md">
```

K3,3: 3 дома и 3 колодца

</div>

```
<p class="text-slate-300 text-sm mb-4">
```

```
 Три дома хотят соединиться с тремя колодцами.
 Нарисовать без пересечений невозможно.
 У него <code class="text-white">V=6, E=9, F=9</code>
 <code class="text-white">9 ≤ 12</code> – проходит, но он не планарен.
 Почему? Потому что у двудольного графа должно быть четное число вершин.
 Отсюда более строгое ограничение: <code class="text-white">9 > 8</code>.
```

</p>

</div>

</div>

```
<details class="bg-slate-900/40 rounded-lg border-rose-500/40 shadow-sm">
 <summary class="p-4 font-bold text-slate-400">Доказательство непланарности K5 (Скрыть)
 <code class="text-slate-300">order-slate-700/50">
```

Доказательство непланарности K5 (Скрыть)

</summary>

```
<div class="p-5 text-sm text-slate-300 space-y-4">
```

```
<p class="text-blue-400">Доказательство непланарности K5
<p>
```

Пусть K5 планарен.  $V = 5$ ,  $E = 10$ .

<bg> $F = E - V + 2 = 10 - 5 + 2 = 7$

<bg>Каждая грань ограничена минимум 3-угольником.

<bg>Сумма длин граней  $= 2E = 20$ .

<bg>Отсюда:  $2E \geq 3F \Rightarrow 20 \geq 21$  – противоречие.

<bg>Значит, K5 не может быть планарным.

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-rose-500 shadow-md">
```

Аналогия: Главный перекресток

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Представь город, где два района соединены перекрестком. Если перекресток исчезнет, то система будет хрупкой (удалят вершину) – районы будут хрупкостью системы.

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border border-emerald-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
```

Телеком: Центральный свитч

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У тебя несколько компьютеров в одной сети, соединенных в звезду (мостом). Сами свитчи – это точки соприкосновения.

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-400">
```

Душный мод: Код и корень DFS (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2">
```

```
<p class="text-blue-400 font-bold">Особый случай
```

```
<p>
```

Для стартовой вершины обхода правил (она не может быть выше неё прыгать некуда). Поэтому для неё не нужно больше 1 независимого ребенка.

```
.....
 Косарайю, свернув каждую SCC в одну супер-вершину (сделаем неориентированным), то любая связность и есть критическая SCC! Если удалить эту вершину. Одно слабое звено изолирует целые касты SCC.

 </div>
</div>
</section>`,
 },
};
```

.....  
ФАЙЛ 17/61: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

```
.....
import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as newChapters } from "../content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
 ...graphChapters,
 ...additionalTickets,
 ...tickets6to13,
 ...tickets14to20,
 ...tickets21to24
].filter(c => c && c.id);

// Remove old 7, 11, 12, 13: graph-planar-colors, graph-bridges
const toRemove = ["graph-planar-colors", "graph-bridges"];

const dedup = new Map<string, Chapter>();
```

```

.....
export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Обзор: Кратчайшие пути и Графы",
 type: "html",
 category: "Графы",
 content: `<section id="intro-content" class="mb-12">
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Обзор
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <p class="text-slate-300 text-sm mb-4">Пред
 оезда). Алгоритмы поиска кратчайшего пути п
 <div class="mt-8 mb-4">
 <div id="slot-mnemonic-cards"></div>
 </div>
 </div>
</div>
</section>`
 },
 {
 id: "dijkstra",
 title: "Билет 14. Дейкстра",
 type: "html",
 category: "Графы",
 content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Дейкс
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-4

```

```

.....

 <div id="slot-dijkstra-viz" class="mt-8"></div>
 </div>
</div>
</section>`
},
{
 id: "bellman-ford",
 title: "Билет 15. Форд-Беллман",
 type: "html",
 category: "Графы",
 content: `<section id="bellman-ford-content" class="
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Форд-Беллман
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500/50">
 <h3 class="text-xl font-bold text-rose-400">Аналогия 1: Параноик

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 еряет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman"
 </div>

```

```
.....
 <div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px; margin-bottom: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Автомагистрали
 </div>
 <p class="text-slate-300 text-sm mb-4">
 евле долететь из Москвы в Париж, если мы сд
 <div id="slot-chapter-image-floyd" style="height: 100px; background-color: #000; margin-top: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
 Время работы
 </div>
 <p class="text-slate-300 text-sm mb-4">
 три вложенных цикла. Сложность $O(V^3)$. Если
 </div>
 </div>

 <div id="slot-floyd-viz" class="mt-8"></div>
 </div>
</div>
</section>`
},
{
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12">
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>

<div class="space-y-8">
```

```

.....
pen:border-slate-700/50">
 Внутренняя структура автомата и код BFS (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Каждая вершина имеет таблицу переходов <code>go</code> по
 ссылке <code>term_link</code> (ближайшая терминальная
 <div class="bg-black/50 p-4 rounded-lg font-mono">
struct Node {
 map<char, int> go; // Предвычисленные переходы
 int link = 0; // Суффиксная ссылка: куда перейти по
 int term_link = 0; // Терминальная ссылка: мат. слово
 bool is_terminal = false;
 vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные link
void buildLinks() {
 queue<int> q;
 // Корни и их дети
 for (auto const& [ch, child] : nodes[0].trie) {
 nodes[child].link = 0;
 q.push(child);
 }

 while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка – это переход по
 nodes[child].link = nodes[nodes[v].link].go[ch];

 // Если суффиксная ссылка сама является словом
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[nodes[child].link].term_link;
 } else {
 nodes[child].term_link = nodes[nodes[child].link].term_link;
 }
 }
 }
}

```

```
export interface QuizQuestion {
 question: string;
 options: string[];
 correctIndex: number;
 explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
 "euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечётными степенями",
 options: [
 "Есть эйлеров цикл (вернись домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)",
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если нечётных больше 2 — эйлеров цикла нет, и эйлеров пути тоже нет."
 },
 {
 question: "Почему гамильтонов цикл — это NP-полная задача?",
 options: [
 "Потому что Гамильтон — болезнь, и лечить её тяжело",
 "Потому что для эйлерова цикла есть простой критерий",
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критерий нечётности степеней. Для гамильтонова цикла нет простого критерия. Это NP-полная задача, которую не известно, можно ли решить за полиномиальное время."
 },
 {
 question: "Куда ведёт эйлеров путь, если нечётных вершин ровно 2?",
 options: [
 "Всё ещё можно пройти по всем рёбрам ровно раз"
```



```

.....
 explanation: "DFS обходит каждую вершину и каждое
 }
],
 "planarity-euler-formula": [
 {
 question: "Планарный граф имеет $V=6$, $E=12$. Возможны ли такие графы?",
 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство $E \leq 3V - 6$ ",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$ — на грани возможности.
 Так что такой граф непланарен."
 },
 {
 question: "Сколько граней у планарного графа с $V=7$ и $E=10$?",
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не может быть 3 грани."
 },
 {
 question: "Почему K_5 непланарен?",
 options: [
 "Потому что $10 > 3*5 - 6 = 9$ — нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин — несчастливое число"
],
 correctIndex: 0,
 explanation: "У K_5 : $V=5$, $E=10$, $3V - 6 = 9$. $10 > 9$ — нарушение."
 }
],
];

```

```
.....
 <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
 Аналогия 1: Корпоративная пре
 </div>
 <p class="text-slate-300 text-sm mb
ы рассылать 10 000 писем, он пишет одно пис
счетах сотрудников только тогда, когда сот
женное обновление).</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Аналогия 2: Матрешки-коробки
 </div>
 <p class="text-slate-300 text-sm mb
ше, и так далее. Если нам нужно покрасить п
Внутри всё синее!" на большую коробку. Опус
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Строгое доказательство / Код (Скр
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
function push(node) {
 if (lazy[node] !== 0) {
 lazy[2 * node] += lazy[node];
 tree[2 * node] += lazy[node];
 lazy[2 * node + 1] += lazy[node];
 tree[2 * node + 1] += lazy[node];
 lazy[node] = 0;
 }
}</pre>

 </div>
```

```

.....
 <div class="absolute -top-3 left-4
 border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 <p class="text-slate-300 text-sm mb-2">
 ужно покрыть отрезок, мы берем две самые бо
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border-
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
 </div>
 </details>
</div>
</div>
</section>`
 },
 {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приоритету",
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">

```

```

.....
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
}
}</pre>
</div>
</details>
</div>
</div>
</section>`
},
{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое самобалансируется",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 (Zig, Zig-Zig, Zig-Zag), гарантируя амортизацию
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: VIP-лифт
 </div>
 <p class="text-slate-300 text-sm mb-4">

```

```

.....
 <div>
 <strong class="text-white block">
 корень?
 <p>Оно поднимет узел, на котором
 ска. За счет сдвига этого листа в корень, д
 /p>
 </div>

 <div>
 <strong class="text-white block">
 <p>Если агент постоянно переключ
 находящимися на разных концах дерева, его
 ащая дерево в вытянутый бамбук для другого.
 . Постоянное свинг-переключение превратит п
 </div>

 <div>
 <strong class="text-white block">
 <p>Хотя один поиск может стоить
 емах обладают прекрасным свойством: они не
 по пути. "Палочное" дерево прессуется в сб
 осов.</p>
 </div>
 </div>
 </div>
 </div>
</section>`
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

.....
 id: "dijkstra",
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">14.
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Аналогия 1
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Позитивное планирование
 <p class="text-xl font-mono text-white">Аналогия 1: Позитивное планирование
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">
 (нельзя поехать и заработать). Он всегда в
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">
 дный и не умеет пересматривать уже "зафиксир
 </div>
 </div>
 <div id="slot-dijkstra-viz" class="mt-8"></div>
 </div>
 </div>
`

```

```

</div>
</div>
<div id="slot-bellman-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="bg-slate-800 p-6 rounded-lg border">
 <div class="absolute -top-3 left-4 bg-slate-700/50
border-emerald-500 shadow-md">
 Суть фокуса (По шагам)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл
 шаге <code class="bg-slate-900 text-pink-400">
 шу себе проходить через вершину k?</i>
 </p>
 <ul class="text-sm text-slate-300 space-y-2">
 Шаг k=0: Ты знаешь только

```

```

.....
 ает очень медленно. Мы хотим сделать Джон
 е пути остались теми же.
 </p>
 <ol class="text-sm text-slate-300 sm:
 Добавляем фиктивную верш
 Запускаем от неё медленно
b>«потенциалы» <code class="text-pink-4
 Меняем старые веса: новое р
 Магия! Все веса тепе
ренние потенциалы сокращаются).
 Теперь смело запускаем б

 </div>
 </div>
 <div id="slot-johnson-viz" class="mt-8"></div>
 </div>
</section>`
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4
 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-emerald-300 italic
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>

```



```
.....
 * Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-6">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </div>
</div>
</div>
</div>
</section>`
},
{
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 rder border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-6">
 изкому племени для объединения. К вечеру пл
```

```
.....
<div class="grid grid-cols-1 xl:grid-cols-2
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb
 Мы идём по строке слева направо.
строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слов
x</code>. Тупой алгоритм начнет искать зано
</code>, а это начало нашего слова! Не надо
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Аналогия 2: LLM стриминг
 </div>
 <p class="text-slate-300 text-sm mb
 Для LLM, которая стримит токены
каждом шаге. Если мы частично совпали с зап
акого места этого слова мы можем продолжить
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
 .b border-slate-700/50">
 Пример вычисления (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <p>Тот же пример: abcbcd</p>
 <ul class="list-disc pl-5 space-y-2
```

```
.....
 type: "html",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 са 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Линейка-шаблон
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Тут мы берём всю строку и «приклеиваю
 я начну читать строку отсюда, насколько длинна
 Это самый быстрый способ найти индекс
 >, считаешь Z-функцию, и там, где <code>Z[i]</code>
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Аналогия 2: LLM Самокопираст
 </div>

```

```
.....
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
```

```
 </div>
```

```
 </details>
```

```
</div>
```

```
</div>
```

```
</section>
```

```
 },
```

```
 {
```

```
 id: "aho-corasick",
```

```
 title: "23. Алгоритм Ахо-Корасик",
```

```
 type: "html",
```

```
 category: "Строки",
```

```
 content: `
```

```
<section id="aho-corasick" class="mb-12 scroll-mt-10">
```

```
 <div class="flex items-center mb-6 flex-wrap gap-3">
```

```

```

```
 <h2 class="text-2xl sm:text-3xl font-bold text-white">
```

```
 </div>
```

```
<div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border-l">
```

```
 <h3 class="text-xl font-bold text-blue-400 mb-4">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-6 text">
```

```
 <p class="text-sm text-blue-300 italic mb-2">Осн
```

```
 <p class="text-lg font-mono text-white">Бор (Тр
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm">Позволяет найти
 го один проход.</p>
```

```
 </div>
```

```
<!-- Аналогии -->
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
```

```
 <div class="bg-slate-800 p-6 rounded-lg border bo
```

```
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
```

```
 Аналогия 1: Паутина (Бор)
```

```

</details>
</div>
</section>`
},
{
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-1">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 i
 rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс NP --
енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 i
 rder border-purple-500 shadow-md">

```

```

.....
<div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-slate-300 italic"
 <p class="text-xl font-mono text-white">
(V + E).</p>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 i
rder border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb
глубо, но проверка "знает ли Вася Петю" мгр
 </div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 i
border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb
т. Оптимально для почти всех задач.</p>
 </div>
 </div>
</div>

{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-400"

 <div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия DFS -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 i
rder border-indigo-500 shadow-md">
 ♂ DFS (Поиск в глубину) = Жадный
 </div>
 <p class="text-slate-300 text-sm mb
 Что делает: Жадный алгор

```

```
.....
<pre class="bg-slate-950 p-3 rounded
```

```
// DFS (Рекурсия)
vector<bool> vis;
vector<vector<int>> adj;
```

```
void dfs(int v) {
 vis[v] = true;
 for (int u : adj[v]) {
 if (!vis[u]) dfs(u);
 }
}
```

```
</pre>
<pre class="bg-slate-950 p-3 rounded
```

```
// BFS (Очередь)
queue<int> q;
q.push(start_node);
vis[start_node] = true;

while(!q.empty()) {
 int v = q.front(); q.pop();
 for (int u : adj[v]) {
 if (!vis[u]) {
 vis[u] = true;
 q.push(u);
 }
 }
}
```

```
</pre>
</div>
```

```
</details>
```

```
</div>
```

```
</div>
```

```
</section>`,
```

```
{,
```

```
{
```

```
 id: "graph-planar-colors",
```

```
 title: "7. Графы. Планарные. Покраска",
```

```
 type: "html",
```

```
 description: "Формула Эйлера и теорема о 4 красках.
```

```
 category: "Графы. Теория",
```

```

.....

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ество компонент.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm mb-4">
 карту – остались ли серые (неисследованные)
 ь через океан – столько и компонент.</p>
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

```



Универсальное пособие • полный исходный код

```
.....
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
 visited[v] = true;
 for (int to : adj[v]) {
 if (!visited[to]) {
 dfs(to);
 }
 }
 ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
}

void topological_sort(int n) {
 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
 reverse(ans.begin(), ans.end()); // Разворачиваем с
}</pre>
```

```
 </div>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая-Жу",
 category: "Графы. Продвинутые",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 10. Графы. Компоненты сильной связности (SCC)
 <h2 class="text-2xl sm:text-3xl font-bold text-white">Графы. Продвинутые</h2>
 </div>
```

```

border-rose-500 shadow-md">
 Связь с Билетом 12
</div>
<p class="text-slate-300 text-sm mb-4">
 SCC (Билет 10) склеивает
</p>

 Точки сочленения (Билет 12)
</p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg p-4 border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Душный мод: Код Алгоритм Косарайю
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-emerald-400 font-bold">Алгоритм Косарайю</p>
 <ol class="list-decimal list-inside">
 Запускаем DFS на исходном графе
 Разворачиваем этот список (топология)
 Переворачиваем все ребра в графе
 Идем по <code>order</code>:

 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает граф"
 }
]

```

```
.....

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.</p>
 <div class="p-3 bg-slate-950 rounded">

 <p class="text-xl font-bold tex">
 <p class="text-xs text-slate-400">
 </div>
 </div>
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Важно: Это работает то
еются к точкам сочленения. То есть,
ег - это тупик со степенью 1).
 </p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2 gap">
 <div class="bg-slate-800 p-6 rounded-lg bor">
 <div class="absolute -top-3 left-4 bg-s
rder-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района сое
её перекроют (удалят вершину) – районы буд
g>хрупкость системы.
```

```

} else {
 dfs(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1)
 IS_CUTPOINT(v);
 ++children;
}
}
if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}

```

</div>

</div>

</details>

<div class="bg-indigo-950/60 p-6 rounded-xl border">

<h4 class="text-lg font-bold text-indigo-400">

<p class="text-slate-300 text-sm mb-4">

Это глубокий дуальный мост между ориентированным графом и неориентированным графом.

</p>

<ul class="list-disc list-inside text-sm text-slate-300">

<li><strong class="text-rose-400">Точки

онг> и показывают физическую уязвимость связности графа.

<li><strong class="text-emerald-400">SCC

ависимые "конгломераты".</li>

<li><strong>Как точка сочленения рушит связность графа?

Косарайю, свернув каждую SCC в одну супер-вершину (сделаем неориентированным), то любая <strong>

о и есть критическая SCC! Если удалить эту вершину, граф распадется.

. Одно слабое звено изолирует целые кластеры SCC.

</li></ul>

</div>

</div>

</section>',

},

{

id: "graph-euler",

title: "13. Графы. Эйлеров цикл",

Универсальное пособие • полный исходный код

.....  
.....  
ФАЙЛ 24/61: src/components/ArticulationPointsViz.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from "react";
import { Play, Pause, RotateCcw, Info } from "lucide-react";
```

```
interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```
interface Edge {
 u: number;
 v: number;
}
```

```
const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];
```

```
const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
```

```

.....
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
};

recordStep("Начало DFS (Тарьян) для поиска точек со

const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

 for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], curren
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${
 v,
 });
 } else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], curren

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}
 v,
);

 if (currentLow[to] >= currentTin[v] && p !==
 ap.add(v);
 }
 }
};

```

```

low: [],
};

useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (

```

```

 });
 }}}

 { /* Nodes */
 { nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id;
 const isVisited = currentStep.visited.has(node.id);
 const isAp = currentStep.ap.has(node.id);

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";
 const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";
 const r = isAp ? 24 : 20;

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-300" />
 <text
 x={node.x}

```



.....

```

 }}
 </g>
);
 }}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-400">
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-slate-800">
 <p className="text-white font-medium min-h-[100px]">
 {currentStep.desc}
 </p>
 </div>

 <div className="space-y-4 flex-1 overflow-y-auto">
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 font-medium">Шаг 1</h5>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-slate-800">
 Точка сочленения
 </div>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-slate-800">
 Посещена
 </div>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-slate-800">
 Текущая
 </div>
 <div className="flex items-center gap-2">
 <div className="w-8 h-1 bg-rose-500 border">
 Мост
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>

```

ФАЙЛ 25/61: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 iteration: number;
 checkingEdge: { u: string; v: string; w: number } | null;
 distances: { [key: string]: number };
 previous: { [key: string]: string | null };
 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}

export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const nodes: Node[] = [
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

 const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 999 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расстояние до А: 999',
 activeCodeLine: 2,
 });

 const vCount = nodes.length;
 for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}. ${
 updated ? 'Релаксация успешна! dist[${edge.v}] = ' + dist[edge.v] : 'Релаксация не удалась'
 }`,
 activeCodeLine: updated ? 6 : 5,
 });
 }

 if (!anyUpdate && iter < vCount - 1) {

```

```

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Зарплата...</div>;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" style={{border: '1px solid #333'}}>
 <div className="flex flex-wrap items-center justify-between" style={{padding: 10px}}>
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-blue-600 text-white rounded-lg">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-blue-300">Полный
 </p>
 </div>
 </div>

 <div className="flex items-center gap-3">
 <div className="bg-slate-900 p-1 rounded-lg border" style={{border: '1px solid #333'}}>
 <button onClick={() => setHasCycle(false)} className="p-2 bg-blue-600 text-white" style={{background: 'linear-gradient(to right, #1e304d, #1e304d)'}}>
 <button onClick={() => setHasCycle(true)} className="p-2 bg-blue-600 text-white" style={{background: 'linear-gradient(to right, #1e304d, #1e304d)'}}>
 </div>
 <div style="display: flex; align-items: center; gap: 10px; padding: 0 10px;">
 <div style="display: flex; align-items: center; gap: 5px;">
 <div style="width: 15px; height: 15px; background-color: #1e304d; border-radius: 50%;">
```

```

.....
 const isCyclePart = hasCycle && ((edge.u === uNode.id && edge.v === 'D'));

 let strokeColor = '#475569';
 let marker = 'url(#arrow-bf-normal)';
 if (isChecking) { strokeColor = '#f59e0b'; }
 else if (currentStep.hasNegativeCycle && isChecking) { strokeColor = '#f43f5e'; }

 return (
 <g key={idx}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} strokeWidth={
 currentStep.hasNegativeCycle && isCyclePart ? 4 : 2} marker={marker} />
 <circle cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)} fill={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#475569'} stroke={strokeColor} />
 <text x={({uNode.x + vNode.x} / 2)} y={({uNode.y + vNode.y} / 2)} fill={strokeColor} fontSize="11" fontWeight="bold" textAnchor="middle" />
 </g>
);
 });
}

(nodes.map((node) => {
 const dist = currentStep.distances[node.id];
 const isCheckingNode = currentStep.checkingNode === node.id;

 return (
 <g key={node.id} className="transition-...>
 <circle cx={node.x} cy={node.y} r={isCheckingNode ? '#7dd3fc' : '#64748b'} strokeWidth={2} />
 <text x={node.x} y={node.y + 5} fill={strokeColor} />
 <text x={node.x} y={node.y - 28} fill={strokeColor} />
 <text x={node.x} y={node.y + 35} fill={strokeColor} />
 </g>
);
}));

```

```

.....
 const d = currentStep.distances[n.id];
 const isTgt = currentStep.checkingEdge
 return (
 <div key={n.id} className={`p-2 rounded
: 'border-slate-700/60 bg-slate-800/40'}`>
 <div className="text-xs text-slate
 <div className={`font-mono font-bo
v>
 </div>
);
 }}}}
 </div>
 </div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
 <div>
 <h4 className="text-lg font-bold text-slate
 <div className="bg-slate-950/80 rounded-lg
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5
e === item.line ? 'bg-blue-900/80 text-amber
 <span className="text-slate-600 w-5 s
 <span className="whitespace-pre flex-
 </div>
)}}
 </div>
</div>
</div>
</div>
</div>
);
}

```

.....

```
// Init
steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0)."});

queue.push(0);
visited.add(0);
steps.push({
 activeLine: 2,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Добавляем стартовую вершину 0 в очередь и по"});

while (queue.length > 0) {
 steps.push({
 activeLine: 4,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: `Очередь не пуста, элементов: ${queue.length}`});

 const curr = queue.shift()!;
 steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`});
}
```

```

.....
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
 });

 return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
 const [stepIdx, setStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const step = STEPS[stepIdx];

 const handleNext = useCallback(() => {
 setStepIdx(prev => Math.min(prev + 1, STEPS.length
 }, []));

 const handlePrev = useCallback(() => {
 setStepIdx(prev => Math.max(prev - 1, 0));
 }, []);

 useEffect(() => {
 if (!isPlaying) return;
 if (stepIdx >= STEPS.length - 1) {
 setIsPlaying(false);
 return;
 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
 }, [isPlaying, stepIdx, handleNext]);

 return (
 <div className="bg-slate-900 border-2 border-slate-
 <div className="flex flex-col md:flex-row items-c
 <div>

```



```

{EDGES.map((e, idx) => {
 const n1 = NODES.find(n => n.id === e.from)
 const n2 = NODES.find(n => n.id === e.to)
 const isVisited = step.visited.includes(n1)
 return (
 <line
 key={idx}
 x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
 className={`transition-all duration-500`}
 >
)
})}
 </>
);
}}}

```

```

{/* Вершины */}
{NODES.map(n => {
 const isCurrent = step.currentNode === n.id
 const isInQueue = step.queue.includes(n.id)
 const isVisited = step.visited.includes(n.id)

 let fill = '#1e293b'; // slate-800
 let stroke = '#475569'; // slate-600
 let scale = 1;

 if (isCurrent) {
 fill = '#3b82f6'; // blue-500
 stroke = '#60a5fa'; // blue-400
 scale = 1.3;
 } else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
 } else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
 }

 return (
 <div
 key={n.id}
 style={{
 position: 'relative',
 width: 40px, height: 40px,
 border: 2px solid stroke,
 background: fill,
 transform: `scale(${scale})`,
 margin: 10px,
 }}
 >

```

```

.....
 { line: 2, code: 'queue.push(start); vi
 { line: 3, code: ' },
 { line: 4, code: 'while not queue.isEmp
 { line: 5, code: ' node = queue.pop()'
 { line: 6, code: ' for neighbor in grap
 { line: 7, code: ' if neighbor not in
 { line: 8, code: ' visited.add(ne
 { line: 9, code: ' queue.push(nei
].map((cl) => {
 const isActive = step.activeLine === cl
 return (
 <div key={cl.line} className={`px-2 p
d border-l-2 border-blue-400 -ml-0.5' : 'te
 <span className="opacity-40 w-6 inl
 {c
 </div>
);
 }}}
</div>
</div>

<div className="bg-slate-800/80 border border
 <div className="text-sm font-bold text-eme
 Состояние Очереди (Queue):
 <span className="text-xs text-slate-500
 </div>
 <div className="flex items-center gap-2 mi
 {step.queue.length === 0 ? (
 <span className="text-slate-500 text-x
) : (
 step.queue.map((qId, i) => (
 <span key={`${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg si
 {qId}

))
)}
</div>

```

```
.....
 alt: 'Все ко Всем Флойд',
 caption: ' Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
};

export default function ChapterImage({ vizType }: { vizType: string }) {
 const info = imageMap[vizType];
 if (!info) return null;

 return (
 <div>
 <div className={`rounded-2xl overflow-hidden border-2 border-emerald-500/30`} >

 </div>
 <p className={`text-center text-xs mt-2 font-bold text-emerald-400`} >{info.caption}</p>
 </div>
);
}
```

ФАЙЛ 28/61: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
.....

import { useState, useEffect, useRef } from "react";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "lucide-react";

interface CodeLine {
 line: string;
 highlight: string;
}

const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " if (!visited[p]) dfs(p, v);", highlight: "normal" },
 { line: " if (p === -1) return true;", highlight: "normal" },
 { line: " return false;", highlight: "normal" },
 { line: "}", highlight: "normal" },
];
```

```

.....
 { id: 5, label: "F", x: 260, y: 370 },
 { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
 { u: 0, v: 1, key: "0-1" },
 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridge
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1
 activeEdge: [1,6], backEdge: null,
 log: "Из B идём в G. tin[G]=3, low[G]=3.",
 activeCodeLine: [4,5,7,10,11]
 },
 {

```

```

.....
 log: "E→F. tin[F]=7, low[F]=7.",
 activeCodeLine: {4,5,7,10,11}
},
{
 currentNode: 5, visitedIndices: {0,1,6,2,3,4,5},
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: {5,3},
 log: "Из F видим D (посещён). Обратное ребро (F,D).",
 activeCodeLine: {7,8,9}
},
{
 currentNode: 4, visitedIndices: {0,1,6,2,3,4,5},
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо",
 activeCodeLine: {11,13,16,17}
},
{
 currentNode: 3, visitedIndices: {0,1,6,2,3,4,5},
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо",
 activeCodeLine: {11,13,16,17}
},
{
 currentNode: 2, visitedIndices: {0,1,6,2,3,4,5},
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – не мо",
 activeCodeLine: {13,14,15,16,17}
},
{
 currentNode: 1, visitedIndices: {0,1,6,2,3,4,5},
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) – не мо",
 activeCodeLine: {11,13,16,17}
},

```



.....

```

 </>
 })
 </g>;
 }}}
</svg>
<div className="flex items-center justify-between" >
 Mar {dfsIdx+1}/{DFS_STEPS.length}
 <div className="flex-1 mx-2 bg-slate-950 h-100px" >
 <div className="bg-indigo-600 h-full transition-all duration-200" >
 <div>
 </div>
 </div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
 {/* Code panel */}
 <div className="bg-slate-950 rounded-lg border border-slate-800" >
 <div className="bg-slate-900 px-3 py-1.5 text-slate-400" >
 <div className="p-2 overflow-x-auto">
 <pre className="text-[10px] font-mono leading-relaxed" >
 {CODE_LINES.map(({cl, i}) => {
 const hl = step.activeCodeLine.includes(cl)
 return <div key={i}
 className={`px-1 transition-all duration-200
 ${hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-400"}>

 {cl.line || ' '}

 </div>;
 })}
 </pre>
 </div>
 </div>
 </div>

 {/* Stack */}
 <div className="bg-slate-900/50 rounded-xl border border-slate-800" >

```

ФАЙЛ 29/61: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React, { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}

export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
 { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
 { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
];

 const edges: Edge[] = [
 { u: 'A', v: 'B', w: 4 },
 { u: 'A', v: 'C', w: 2 },
 { u: 'C', v: 'B', w: 1 },
 { u: 'C', v: 'G', w: 3 },
 { u: 'C', v: 'E', w: 8 },
 { u: 'B', v: 'G', w: 4 },
 { u: 'B', v: 'D', w: 5 },
 { u: 'G', v: 'D', w: 1 },
];
```



```

.....
 log: ` Дейкстра готов к доставке. Начинаем из ПИ
 activeCodeLine: 2,
 });

for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }

 if (!u) break;

 visited[u] = true;
 simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Выбираем вершину ${u} с мин. непосещённой
 activeCodeLine: 5,
 });

 const neighbors = edges.filter(e => e.u === u);
 for (const edge of neighbors) {
 const v = edge.v;
 if (visited[v]) continue;

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Смотрим соседа ${v}. Текущее расстояние
 ${dist[u] + edge.w}.`,
 activeCodeLine: 8,
 });

 if (dist[u] + edge.w < dist[v]) {

```

```

.....
return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 {/* Шапка симулятора */}
 <div className="flex flex-wrap items-center justify-between"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white"

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Полный
 </div>
 </div>

 <div className="flex items-center gap-2">
 <button
 onClick={() => { setIsPlaying(false); setCurrentStepIndex(0); }}
 className="flex items-center gap-1 bg-slate-700 text-white px-4 py-2 rounded-lg"
 >
 Сброс
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1 px-4 py-2 rounded-lg ${isPlaying ? 'bg-amber-600 hover:bg-amber-700' : 'bg-slate-700 hover:bg-slate-600'}`}
 >
 {isPlaying ? 'Пауза' : '▶ Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); if (currentStepIndex < steps.length - 1) { setCurrentStepIndex(currentStepIndex + 1); } }}
 disabled={currentStepIndex >= steps.length - 1}
 className="flex items-center gap-1 bg-indigo-600 text-white px-3 py-2 rounded-lg text-sm font-medium transition-colors"
 >
 Шаг
 </button>
 </div>
 </div>
 </div>
);

```

```

.....
 return (
 <g key={idx}>
 <line
 x1={uNode.x} y1={uNode.y}
 x2={vNode.x} y2={vNode.y}
 stroke={isChecking ? '#f59e0b' : isOpt
 strokeWidth={isChecking ? 5 : isOpt
 markerEnd={marker}
 className="transition-all duration-
 strokeDasharray={isChecking ? '4,4'
 />
 <circle
 cx={({uNode.x + vNode.x) / 2} cy={({u
 r="12" fill="#1e293b"
 stroke={isChecking ? '#f59e0b' : '#f
 />
 <text
 x={({uNode.x + vNode.x) / 2} y={({uNo
 fill={isChecking ? '#f59e0b' : '#f8
 fontSize="11" fontWeight="bold" tex
 >
 {edge.w}
 </text>
 </g>
);
 }}}

 {/* Вершины */}
 {nodes.map((node) => {
 const isCurrent = currentStep.currentNode
 const isVisited = currentStep.visited[node.i
 const dist = currentStep.distances[node.i

 return (
 <g key={node.id} className="transition-
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}

```

```

 te-300'}`}>
 {u}

 →
 </div>
 <div className="flex flex-wrap gap-2" >
 {adjList[u].map((edge, idx) => {
 const isCheckingThis = currentStep === idx
 return (
 <span key={idx} className={`px-2 py-1
 isCheckingThis ? 'bg-amber-500' : 'bg-white'
 600 text-slate-300'`} >
 {edge.v}

 {edge.w}

)
 })}
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>

 <div className="flex flex-col lg:flex-row gap-6">
 <div>
 <h4 className="text-lg font-bold text-rose-950">Таблица расстояний
 <div className="overflow-x-auto">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-rose-950">
 <th className="py-2 px-3">Вершина</th>
 <th className="py-2 px-3">Расстояние</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <td>A</td>
 <td>0</td>
 </tr>
 <tr>
 <td>B</td>
 <td>1</td>
 </tr>
 <tr>
 <td>C</td>
 <td>2</td>
 </tr>
 <tr>
 <td>D</td>
 <td>3</td>
 </tr>
 <tr>
 <td>E</td>
 <td>4</td>
 </tr>
 <tr>
 <td>F</td>
 <td>5</td>
 </tr>
 <tr>
 <td>G</td>
 <td>6</td>
 </tr>
 <tr>
 <td>H</td>
 <td>7</td>
 </tr>
 <tr>
 <td>I</td>
 <td>8</td>
 </tr>
 <tr>
 <td>J</td>
 <td>9</td>
 </tr>
 </tbody>
 </table>
 </div>
 </div>
 </div>

```

```

.....
 {codeLines.map((item) => {
 const isActive = currentStep.activeCodeLine === item.line
 return (
 <div key={item.line} className={`py-1
 isActive ? 'bg-indigo-900/80 text-white' : ''}>
 {item.line}
 {item.code}
 </div>
)
 })}
 </div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 30/61: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

.....

import { useState, useEffect } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'return';
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}

```

```

.....
 desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано
 codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
}

if (n <= 2) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возврат
 codeLine: 3,
 memoState: { ...memoMap }
 });
 return 1;
}

generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вычисляем рекурсивно: fibМемо(${n - 1})
 codeLine: 4,
 memoState: { ...memoMap }
});

const res1 = simulateFib(n - 1);
const res2 = simulateFib(n - 2);
memoMap[n] = res1 + res2;

generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: мемо[${n}] = ${res1} +
 codeLine: 5,

```

```

.....
<div className="bg-slate-900 rounded-2xl border border-1
 {/* Header & Controls */}
 <div className="flex flex-col lg:flex-row lg:items-center"
 <div>
 <div className="flex items-center gap-3 mb-2">
 <div className="p-2 bg-emerald-500/10 border border-1"
 <RefreshCw className="w-6 h-6" />
 </div>
 <h3 className="text-2xl font-extrabold text-white">
 </div>
 <p className="text-sm text-slate-400">
 Смотри, как таблица DP кэширует результаты
 </p>
 </div>

 <div className="flex flex-wrap items-center gap-3"
 <div className="flex items-center gap-2 bg-slate-900"
 N:
 <select
 value={targetN}
 onChange={(e) => setTargetN(Number(e.target.value))}
 disabled={isPlaying}
 className="bg-transparent text-white font-mono"
 >
 <option value={4}>N = 4</option>
 <option value={5}>N = 5</option>
 <option value={6}>N = 6</option>
 <option value={7}>N = 7</option>
 </select>
 </div>

 <div className="flex items-center bg-slate-950"
 <button
 onClick={() => { setIsPlaying(false); setTargetN(4); }}
 className="p-2 text-slate-400 hover:text-white"
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />

```

```

.....
 const val = num <= 2 ? 1 : memoState[num];
 const isCurrent = currentStep?.n === num;

 return (
 <div
 key={num}
 className={`flex flex-col items-center
 isCurrent
 ? 'bg-emerald-950 border-emerald-500
 : isCached
 ? 'bg-slate-900 border-emerald-500/50
 : 'bg-slate-900/50 border-slate-800
 `}
 >

 <span className={`text-xl md:text-2xl font-medium
 {isCached ? val : '0'}
 >
 {num <= 2 &&
 {currentStep?.desc}
 }

 <div className="text-xs font-mono bg-slate-900/50 border-slate-800 p-2">
 Текущий N: <strong className="text-emerald-400">{currentStep?.n}
 </div>
 </div>
);
 }
}
</div>

{/* Tabs */}

```



```

.....
 Всего шагов симуляции: {steps.length}
 <span className="text-emerald-400 font-bo
 </div>
 </div>
})

{activeTab === 'code' && (
 <div className="bg-slate-950 rounded-2xl bord
 <div className="bg-slate-900 px-6 py-3 bord
 <span className="text-xs font-bold font-m
 <span className="text-xs text-emerald-400
 </div>
 <pre className="p-6 font-mono text-sm space
 {DP_CODE_LINES.map(({line, idx}) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine
 return (
 <div
 key={lineNum}
 className={`flex items-center px-4
 isActive
 ? 'bg-emerald-600/20 border-l-4
 : 'text-slate-300 hover:bg-slate
 `}>
 <div>
 <span className="w-8 text-xs text-s
 {line}
 </div>
 </div>
)};
)})}
 </pre>
 <div className="p-6 bg-slate-900/50 border-
 <span className="p-1.5 bg-emerald-500/20
 {currentStep?.desc}
 </div>
</div>
})
</div>

```

```

.....
 { u: 1, v: 2 },
 { u: 3, v: 4 }
];

export function EulerSimulator() {
 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Клики на вершину
 const [c1Done, setC1Done] = useState(false);

 const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
 };

 const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы п
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v
 if (!edge) {
 setC1Msg("Нет ребра между этими вершинами! Выбери
 return;
 }
 const key = `${Math.min(last, vId)}-${Math.max(last,
 if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощае
 setC1Done(true); return;
 }
 const nextEdges = [...c1VisitedEdges, key];
 const nextPath = [...c1Path, vId];
 setC1VisitedEdges(nextEdges);
 setC1Path(nextPath);

 if (nextEdges.length === C1_EDGES.length) {

```

```

.....
 {C1_NODES.map(n => {
 const isLast = c1Path[c1Path.length-1] === n.id;
 const visited = c1Path.includes(n.id);
 return <g key={n.id} className="cursor-point"
 {isLast && <circle cx={n.x} cy={n.y} r={11}
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
 stroke={isLast ? "#fff" : visited ? "#6366f1" : "#fff"}
 strokeWidth={2.5}
 className="transition-all duration-200"
 <text x={n.x} y={n.y+4} textAnchor="middle">{n.label}</text>
 </g>;
 })}
 </svg>
 <div className="absolute top-2 left-2 bg-slate-500 p-2"
 >Решено: {c1VisitedEdges.length}/{C1_EDGES.length}</div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
 c1Done && c1VisitedEdges.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
 : c1Done
 ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
 : "bg-slate-900/50 border-slate-700 text-slate-50"}
 >
 {c1Msg}
</div>
</div>
);
}

```

```
.....
 explanation: 'Именно! Дейкстра уверен: если ты прие
 ля чего нужен алгоритм Беллмана-Форда.'
},
{
 id: 3,
 question: 'Какова ключевая фишка алгоритма Борувки
 options: [
 'Он использует меньше памяти благодаря коммунистиче
 'Он умеет работать с отрицательными весами без ци
 'Он позволяет выполнять шаги слияния колхозов (ко
 'Он был разработан в Токио и поэтому самый быстрый
],
 correctAnswer: 2,
 explanation: 'Абсолютно верно! В Борувке каждая ком
 лелить вычисления на GPU или многопроцессорных кл
},
{
 id: 4,
 question: 'В чем главная суть Динамического програм
 options: [
 'Писать код очень быстро, динамично нажимая на кл
 'Запоминать (кешировать) результаты уже решенных
 'Использовать динамическую типизацию как в Python
 'Постоянно менять требования к проекту в процессе
],
 correctAnswer: 1,
 explanation: 'Точно! Главный принцип ДП – мемоизаци
},
{
 id: 5,
 question: 'Какая структура данных идеально помогает
 options: [
 'СНМ (Система непересекающихся множеств / Disjoin
 'Стек (LIFO).',
 'Красно-черное дерево.',
 'Хэш-таблица со списками коллизий.'
],
 correctAnswer: 0,
```

```

 setShowResults(false);
 };

 if (showResults) {
 return (
 <div className="bg-slate-900/90 border border-slate-
 my-12">
 <div className="w-20 h-20 bg-gradient-to-tr from-
 -6 shadow-xl shadow-indigo-500/30">
 <Award className="w-10 h-10 text-white" />
 </div>
 <h3 className="text-3xl font-bold text-white mb-
 <p className="text-slate-400 text-sm mb-6">Ты п

 <div className="bg-slate-950 border border-slate-
 <p className="text-slate-400 text-sm uppercase
 <p className="text-5xl font-black text-indigo
 <p className="text-slate-300 text-sm">
 {score === quizQuestions.length
 ? ' Идеально! Ты настоящий сеньор-раскра
 : score >= 3
 ? ' Отличный результат! Главные концепции
 : ' Стоит повторить главы пособия и еще
 </p>
 </div>

 <button
 onClick={handleRestart}
 className="inline-flex items-center gap-2 px-4
 d rounded-xl shadow-lg shadow-indigo-600/30
 >
 <RotateCcw className="w-5 h-5" />
 Пройти тест заново
 </button>
 </div>
);
}

```

```

.....
 key={idx}
 onClick={() => handleSelectOption(idx)}
 className={"flex items-start gap-4 p-4"}
 >
 <div className="mt-0.5 shrink-0">
 {isAnswered && isCorrect ? (
 <CheckCircle2 className="w-6 h-6 text-emerald-500" />
) : isAnswered && isSelected && !isCorrect ? (
 <XCircle className="w-6 h-6 text-rose-500" />
) : (
 <div className={"w-6 h-6 rounded-full border border-indigo-500 bg-indigo-500 text-white"}>
 {String.fromCharCode(65 + idx)}
 </div>
)}
 </div>
 <div className="flex-1 text-sm md:text-base">
 {option}
 </div>
 </div>
);
}
}
</div>
</div>

{isAnswered && (
 <div className="bg-slate-950 border border-slate-800 p-4">
 <div className="flex items-center gap-2 mb-2">
 <AlertCircle className="w-4 h-4" />
 Объяснение:
 </div>
 <p className="text-slate-300 text-sm leading-relaxed">
 {currentQ.explanation}
 </p>
 </div>
)}

<div className="flex justify-end pt-4 border-t border-slate-800">

```

```

.....
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

const adjList: { [key: number]: { v: number; w: number }[] } = {
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
 2: [{ v: 4, w: 2 }],
 3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
 4: [{ v: 6, w: 1 }],
 5: [{ v: 6, w: 5 }],
 6: [{ v: 1, w: 1 }],
};

const codeLines = [
 { line: 1, text: 'function floyd_warshall(matrix, V' },
 { line: 2, text: ' dist = copy(matrix)' },
 { line: 3, text: ' for k from 0 to V - 1: // Пос' },
 { line: 4, text: ' for i from 0 to V - 1:' },
 { line: 5, text: ' for j from 0 to V - 1' },
 { line: 6, text: ' if dist[i][k] + d' },
 { line: 7, text: ' dist[i][j] = ' },
 { line: 8, text: ' return dist' },
];

```

```

.....

simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ' Все пары отработаны. Алгоритм Флойда завершён',
 activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-emerald-500">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded-md">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">Алгоритм Флойда</h3>
 <p className="text-sm text-emerald-300">Получить матрицу</p>
 </div>
 </div>
 </div>
 </div>
);

```



```

.....
 return (
 <g key={`${u}-${edge.v}`}>
 <line x1={uNode.x} y1={uNode.y} x2=
markerEnd={marker} strokeDasharray={isTarget
 <circle cx={({uNode.x + vNode.x) / 2}
isVia ? '#818cf8' : '#64748b'} strokeWidth=
 <text x={({uNode.x + vNode.x) / 2} y=
8fafc'} fontSize="10" fontWeight="bold" tex
 </g>
);
 });
 });
 {nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748b';
 if (isK) { fill = '#4f46e5'; stroke = '#a1887f'; }
 else if (isI) { fill = '#b45309'; stroke = '#c07040'; }
 else if (isJ) { fill = '#047857'; stroke = '#2e8b57'; }
 return (
 <g key={node.id} className="transition-colors">
 <circle cx={node.x} cy={node.y} r={isTarget
sJ ? 4 : 2} className="transition-colors duration-200">
 <text x={node.x} y={node.y + 5} fill="white">
 <text x={node.x} y={node.y + 32} fill="white">
 </text>
 </g>
);
 });
}
</svg>

<div className="w-full bg-slate-950/80 p-4 rounded">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
</div>
</div>

```

```

</div>
<div className="overflow-x-auto">
 <table className="w-full text-center border">
 <thead><tr className="bg-rose-950/40 text">
 <th className="p-2 border-r border-rose">
 {nodeNames.map((n, idx) => <th key={idx}
-400 font-bold' : ''}`}>{n.split(' ')[0]} {
 </tr></thead>
 <tbody className="text-xs font-mono">
 {currentStep.matrix.map((row, i) => (
 <tr key={i} className="border-b borde
 <td className={`p-2 bg-rose-950/40
tStep.k === i ? 'text-amber-400' : ''}`}>
 {nodeNames[i].split(' ')[0]} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i
 const isVia = currentStep.k !==
tep.j === j});
 let bg = 'bg-slate-900/40 text-sla
 if (isUpd) bg = 'bg-emerald-900/8
 else if (isVia) bg = 'bg-indigo-9
 else if (i === j) bg = 'bg-slate-
 return (<td key={j} className={`p
 /td>);
 }}}
 </tr>
 }}}
 </tbody>
 </table>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60">
 <div>
 <h4 className="text-lg font-bold text-slate">
 <div className="bg-slate-950/80 rounded-lg

```

```

.....
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];

const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F', 'G'],
 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};

type Action = {
 type: 'visit' | 'process' | 'backtrack';
 node: string;
 desc: string;
 queueOrStack?: string[];
 visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const stack: string[] = []; // for visualization, j

 function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Зависит от`,
 visitedNodes: Array.from(vis) });

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 steps.push({ type: 'process', node: v, desc: `Процесс`,
 queueOrStack: stack, visitedNodes: Array.from(vis) });
 }
 }
 }
}

```

```

 steps.push({ type: 'backtrack', node: '', desc: `04
 om(vis)} }));

 return steps;
 }

export function GraphTraversalViz() {
 const [mode, setMode] = useState<"dfs" | "bfs">("df
 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);

 const steps = useMemo(() => mode === 'dfs' ? genera

 useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
 }, [mode]);

 useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 }, 1200);
 return () => clearInterval(t);
 }
 }, [autoPlay, steps.length]);

 const step = steps[stepIdx] || steps[0];

 const isVisited = (nodeId: string) => step.visitedN

```

```

.....

 const visited = isVisited(n)
 const current = isCurrent(n)

 let fillColor = "fill-slate"
 let strokeColor = "stroke-slate"
 let textColor = "fill-slate"
 let radius = 18;

 if (visited) {
 fillColor = mode === 'dark' ? "fill-white" : "fill-slate"
 strokeColor = mode === 'dark' ? "stroke-white" : "stroke-slate"
 textColor = "fill-white"
 }

 if (current && step.type !== 'edge') {
 strokeColor = "stroke-amber"
 textColor = "fill-amber"
 radius = 22;
 }

 return (
 <g key={n.id} className={className} >
 <circle cx={n.x} cy={n.y} radius={radius}
 className={`stroke-${strokeColor} fill-${fillColor}`} />
 <text x={n.x} y={n.y} text={n.label}
 className={`text-${textColor}`} />
 </text>

 {
 /* Ripple for BFS */
 current && mode === 'bfs' && !visited && !current
 ? <circle cx={n.x} cy={n.y} radius={22}
 className="stroke-amber fill-transparent" />
 : null
 }
 </g>
)
 }
}
</svg>

```

```

 </button>
 <button onClick={() => setAutoPlay(
justify-center min-w-[80px] ${mode === 'df
>
 {autoPlay ? <><Pause size={16}
 </button>
 <button onClick={() => {setAutoPlay
= steps.length - 1} className="p-2 text-sla
hover:bg-transparent">
 <SkipForward strokeWidth={3} si
 </button>
 <button onClick={() => {setAutoPlay
g-slate-800 rounded-lg ml-2 border-1 border
 <RefreshCw strokeWidth={3} size
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50
ow-hidden">
 <div className="text-[10px] text-sl
{steps.length}</div>
 <div className="text-sm text-slate-
 {step.desc}
 </div>
 </div>
</div>
</div>
);
}

```

ФАЙЛ 35/61: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useCallback } from 'react';
```

```

 } else break;
 }
 return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient {
 return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '🫀' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🔪' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '🫀' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🔪' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];
 arr.push(...start);
 return arr;
})();

```

```

.....
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
 });

 const nextHeap = heapInsert(heap, newPatient).map(x => {
 // Mark specifically this new patient as 'in' in the heap
 const target = nextHeap.find(x => x.priority === pr
 const marked = nextHeap.map(x => x.id === (target?.

 setHeap(marked);
 addLog(` Поступил пациент: ${finalName} с тяжестью

 setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
 }, 500);
}, [busy, heap]);

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() *
 const randomShift = Math.floor(Math.random() * 10)
 const prio = Math.max(10, Math.min(99, preset.priority - randomShift));
 insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 }
}

```



```

 }, 650));
 }, {busy, heap}));

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 })));
 setLog([' База данных скорой помощи перезагружена.']);
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 }))), 650);
};

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < heap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < heap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 return res;
});

```

```

.....
 onChange={e => setCustomName(e.target.value)}
 placeholder="Имя пациента"
 className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:
 />
 <div className="flex flex-col items-center min-w-[300px] gap-4">
 Приоритет
 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e.target.value))}
 className="w-20 accent-emerald-500 cursor-pointer"
 />
 </div>
 <button
 type="submit"
 disabled={busy || !customName.trim() || heap.length === 0}
 className="bg-teal-600 hover:bg-teal-500 disabled:bg-slate-800 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all"
 >
 Вести
 </button>
 </form>

 {/* Забрать самого тяжелого */}
 <button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-mono transition-all cursor-pointer flex items-center justify-center gap-2"
 >
 В реанимацию (EXTRACT MAX)
 </button>

 <button

```



```

.....
 {idx === 0 ?
 {entry}
 </div>
)})
 </div>
</div>
);
};

```

ФАЙЛ 36/61: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

.....

import React, { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';

export function JohnsonViz() {
 const [step, setStep] = useState(0);
 const MAX_STEP = 7;

 const nodes = [
 { id: 'S', x: 200, y: 150, label: 'S' },
 { id: 'A', x: 200, y: 50, label: 'A' },
 { id: 'B', x: 320, y: 220, label: 'B' },
 { id: 'C', x: 80, y: 220, label: 'C' },
];

 const edges = [
 { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: -10, newWeight: 0 },
 { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 10, newWeight: 0 },
 { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0, newWeight: 0 },
 { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 1, labelDx: 0, labelDy: 0 },
 { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 2, labelDx: 0, labelDy: 0 },
 { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 3, labelDx: 0, labelDy: 0 },
];

```

```

 };

 const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#a
 if (e.id === 'AB' && step > 4) return "url(#arr
 if (e.id === 'BC' && step === 5) return "url(#a
 if (e.id === 'BC' && step > 5) return "url(#arr
 if (e.id === 'CA' && step === 6) return "url(#a
 if (e.id === 'CA' && step > 6) return "url(#arr

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
 };

 const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

 if (e.id === 'AB') {
 if (step === 4) return `${e.weight} + (${0}
 return e.newWeight;
 }
 if (e.id === 'BC') {
 if (step < 5) return e.weight;
 if (step === 5) return `${e.weight} + (${-2}
 return e.newWeight;
 }
 if (e.id === 'CA') {
 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${-1}
 return e.newWeight;
 }
 };

```

ных вершин:

```


 </div>
);
case 3: return (
 <div>
 Шаг 3: Магическая формула.

 Всем ребрам графа назначаем новый вес.

 <div className="text-center font-monospace" style="border: 1px solid black; padding: 5px; margin: 10px 0;">
 Это повышает или понижает вес ребра?
 </div>
 </div>
);
case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра A → B.

 Старый вес: 0
 Вычисляем: $-2 + 0 - (-2) =$ -2
 </div>
);
case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра B → C.

 Старый вес: 1.

 Вычисляем: $1 + (-2) - (-1) =$ -1
 </div>
);
case 6: return (
 <div>
 Шаг 3.3: Пересчет ребра C → A.

 Старый вес: 2.

 Вычисляем: $2 + (-1) - 0 =$ 1
 </div>
);
case 7: return (
 <div className="text-emerald-300">
 Готово! Фиктивную вершину S мы добавили в граф.
 Все новые веса ребер в графе стали отрицательными.
 При этом старые кратчайшие пути остались прежними.
 </div>
);

```

```

</marker>
</defs>

{/* Edges */}
{edges.map((e, i) => {
 const u = nodes.find(n => n
 const v = nodes.find(n => n

 const isS = e.u === 'S';
 if (isS && (step === 0 || s

 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

 const edgeColorClass = getE
 const textHigh = isEdgeText
 const textEmer = isEdgeText

 return (
 <g key={i}>
 <line
 x1={u.x} y1={u.y}
 className={`str
 markerEnd={getM
 />

 <text
 x={midX} y={midY}
 textAnchor="mid
 dominantBaselin
 className={`tex
 ${textHigh
 textEmer
 isS ? 'fi
 `}>
 {getEdgeContent
 </text>
 </g>
)}

```

```

<div className="w-full md:w-[320px] flex
 {/* Log Box */}
 <div className="bg-[#0f172a] p-5 rounded
 justify-center leading-relaxed text-sm text-white
 {getDesc()}
 </div>

 {/* Controls */}
 <div className="flex bg-slate-900 border
 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Сбросить"
 <RotateCcw strokeWidth={3} fill="white" />
 </button>
 <button onClick={() => setStep(2)}
 className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Варьировать"
 <Undo strokeWidth={3} size={24} fill="white" />
 </button>
 <button onClick={() => setStep(3)}
 className={`flex-1 font-bold text-white
 ${step === MAX_STEP
 ? 'bg-emerald-600/50'
 : 'bg-indigo-600 hover:bg-indigo-500'}
 {step === MAX_STEP ? "Готово" : "В процессе"}
 {step !== MAX_STEP && <Skip />
 </button>
 </div>

 {/* Progress Dots */}
 <div className="flex justify-center gap-2
 {Array.from({length: MAX_STEP + 1}, (_, i) =>
 <div key={i} className={`w-4 h-4 rounded-full
 ${i < step ? 'bg-indigo-900' : 'bg-slate-700'}
 >
 </div>
 </div>
</div>

```



```
.....
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (See 1)
};
```

```
const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];
```

```
interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "finish";
 desc: string;
 visited: Set<number>;
 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}
```

```
export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);
```

```
 useEffect(() => {
 // Generate step-by-step Kosaraju steps
 const generatedSteps: AlgoStep[] = [];
 const visited = new Set<number>();
 const order: number[] = [];
 const sccMap: Record<number, number> = {};
```

```
 const recordStep = (
```

```

.....
 `DFS 1: зашли в вершину ${u}, помечаем посещение
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Запи
 u,
 false,
 -1,
);
 };

 // Run DFS1 on all unvisited
 for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 dfs1(i);
 }
 }

 const topoOrder = [...order].reverse();
 recordStep(
 "transpose",
 `Первый проход завершен! Стек выхода в топологиче
 ебра.` ,
 null,
 true,
 -1,
);

```

```

);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в
 u,
 true,
 sccId,
);
 }
}

recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонент,
 null,
 true,
 sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
 phase: "init",
 desc: "Загрузка...",
 visited: new Set(),
 currentNode: null,
 order: [],
 transposed: false,
 sccMap: {},
 currentSccId: -1,
 activeEdges: initialEdges,
};

useEffect(() => {

```

```

.....
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center
ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] rela
 <svg className="w-full h-full max-w-[650px]" >
 {/* Markers for directed arrows */}
 <defs>
 <marker
 id="arrow"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
 </marker>
 <marker
 id="arrow-active"
 viewBox="0 0 10 10"
 refX="22"

```

```

.....
 if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46"
 markerId = "arrow-transposed";
 } else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
 }

 // Adjust slightly for bidirectional link
 const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
 const dy = isBidi ? (edge.u === 3 ? -10 :

 return (
 <line
 key={`edge-${idx}`}
 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 stroke={stroke}
 strokeWidth="2.5"
 markerEnd={`url(#${markerId})`}
 className="transition-all duration-300"
 />
);
 }}}

 {/* Nodes */}
 {initialNodes.map((node) => {
 const isCurrent = step.currentNode === node
 const isVisited =

```

```

.....
 y={node.y + 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
 })
 </g>
};
}}
</svg>

```

```

{step.transposed && (
 <div className="absolute top-4 left-4 bg-in
 1 rounded">
 Граф транспонирован (Ребра обратные)
 </div>
)}
</div>

```

```

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
 00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-4
 Порядок обхода и стек
 </h4>

```

```

<div className="bg-slate-900 border border-sla
 <p className="text-xs text-white leading-re
 {step.desc}
 </p>
</div>

```

```

<div className="flex-1 space-y-4 overflow-y-a
 {/* Top-Sort output / Order stack represent
 <div>

```

```

 </div>
 </div>
 </div>

 <div className="mt-4 pt-3 border-t border-slate-500">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-500 px-2 py-1 rounded"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-500 px-2 py-1 rounded"
 >
 Вперед
 </button>
 </div>
 </div>
 </div>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 38/61: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from 'react';
```

```

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
 const [activeAlgo, setActiveAlgo] = useState<'kruskal'>('kruskal');
 const [vertices, setVertices] = useState<Vertex[]>([0, 1, 2, 3, 4, 5, 6, 7, 8]);
 const [edges, setEdges] = useState<Edge[]>(initialEdges);
 const [stepIndex, setStepIndex] = useState<number>(0);
 const [heapQueue, setHeapQueue] = useState<string[]>([]);
 const [activeCheatTab, setActiveCheatTab] = useState<'kruskal'>('kruskal');

 const [logs, setLogs] = useState<StepLog[]>([
 {
 title: 'Введение в раскраску (Краскал)',
 description: '«Краскал – это раскраска?». Представим, что мы хотим начать!',
 type: 'info',
 },
]);

 // --- KRUSKAL STEPS ---
 const kruskalSteps = [
 // Step 1: Inspect A-B (2)
 () => {

```



```

.....
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и
 type: 'info'
 }, ...prev]));
 },
// Step 6: Include B-C
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро B-C в осто́ве',
 description: 'Вершина C перекрашена в красный ц
 type: 'success'
 }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Берем ребро A-D (вес 5)',
 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю – мы перекрашиваем всё в один цвет! "',
 type: 'warning'
 }, ...prev]));
},
// Step 8: Include A-D (Merge to purple)
() => {
 setVertices(prev => prev.map(v => v.color === 'bl
 setEdges(prev => prev.map(e => e.status === 'incl
 setLogs(prev => [{
 title: 'Слияние кланов: Ребро A-D добавлено',
 description: 'Мы объединили красный и синий клан
 type: 'success'
 }, ...prev]));
},
// Step 9: Inspect B-E (6) - Cycle!

```

```

.....
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.'
 type: 'success'
 }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Берем ребро C-F (вес 9)',
 description: 'Оно соединяет вершины C и F, кото
 type: 'warning'
 }, ...prev]));
},
// Step 16: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро C-F.',
 type: 'error'
 }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {

```

```

.....
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю I',
 type: 'info'
 }, ...prev]);
},
// Step 22: Include H-I
() => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: ' Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фиолетовый цвет.
 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]);
},
};

// --- PRIM STEPS ---
const primSteps = [
 // Step 1: Start at E
 () => {
 setVertices(prev => prev.map(v => v.id === 4 ? {
 setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-G (8)'],
 setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5', '4-6', '4-7', '4-8']),
 setLogs(prev => [{
 title: 'Шаг 1: Плесень зарождается в Токио (Вершина E)',
 description: 'Мы выбрали вершину E как стартовую вершину. Добавили в приоритетную очередь (Heap): D-E(3), B-E(6), E-F(7), E-G(8).',
 type: 'info'
 }, ...prev]);
 },
 // Step 2: Pop D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Куча выдает самое дешевое ребро D-E (3)',
 description: 'Плесень тянется по ребру D-E к вершине D (вес 3).',
 type: 'info'
 }, ...prev]);
 },
 // Step 3: Add D
 () => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Добавили вершину D. Куча теперь содержит: B-E(6), E-F(7), E-G(8).',
 description: 'Ребро D-E (3) было добавлено. Теперь очередь содержит: B-E(6), E-F(7), E-G(8).',
 type: 'info'
 }, ...prev]);
 },
 // Step 4: Pop B-E (6)
 () => {
 setEdges(prev => prev.map(e => e.id === 'B-E' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает следующее самое дешевое ребро B-E (6)',
 description: 'Плесень тянется по ребру B-E к вершине B (вес 6).',
 type: 'info'
 }, ...prev]);
 },
 // Step 5: Add B
 () => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === 'B-E' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Добавили вершину B. Куча теперь содержит: E-F(7), E-G(8).',
 description: 'Ребро B-E (6) было добавлено. Теперь очередь содержит: E-F(7), E-G(8).',
 type: 'info'
 }, ...prev]);
 },
 // Step 6: Pop E-F (7)
 () => {
 setEdges(prev => prev.map(e => e.id === 'E-F' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает следующее самое дешевое ребро E-F (7)',
 description: 'Плесень тянется по ребру E-F к вершине F (вес 7).',
 type: 'info'
 }, ...prev]);
 },
 // Step 7: Add F
 () => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === 'E-F' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Добавили вершину F. Куча теперь содержит: E-G(8).',
 description: 'Ребро E-F (7) было добавлено. Теперь очередь содержит: E-G(8).',
 type: 'info'
 }, ...prev]);
 },
 // Step 8: Pop E-G (8)
 () => {
 setEdges(prev => prev.map(e => e.id === 'E-G' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает следующее самое дешевое ребро E-G (8)',
 description: 'Плесень тянется по ребру E-G к вершине G (вес 8).',
 type: 'info'
 }, ...prev]);
 },
 // Step 9: Add G
 () => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === 'E-G' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Добавили вершину G. Куча теперь пуста.',
 description: 'Ребро E-G (8) было добавлено. Теперь очередь пуста. MST построен!',
 type: 'info'
 }, ...prev]);
 },
 // Step 10: Final MST
 () => {
 setLogs(prev => [{
 title: 'Шаг 10: Итоговый MST построен!',
 description: 'Итоговый MST состоит из ребер: D-E (3), B-E (6), E-F (7), E-G (8), H-I (12). Общий вес: 30.',
 type: 'info'
 }, ...prev]);
 },
 // Step 11: End
 () => {
 setLogs(prev => [{
 title: 'Шаг 11: Завершение алгоритма Краскала!',
 description: 'Алгоритм Краскала успешно завершил работу. Итоговый MST построен.',
 type: 'info'
 }, ...prev]);
 },
];

```

```

.....
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к вершине A.',
 type: 'info'
 }, ...prev]));
},
// Step 7: Include A-B, add B's edges
() => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));
 setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)']);
 setLogs(prev => [{
 title: 'Успех: Вершина B захвачена плесенью',
 description: 'В кучу добавлено B-C(4). Ребро B-E(6) не добавлено, так как образует цикл.',
 type: 'success'
 }, ...prev]));
},
// Step 8: Pop B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
 description: 'Плесень тянется к вершине C.',
 type: 'info'
 }, ...prev]));
},
// Step 9: Include B-C, add C's edges
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 eap' } : e));
 setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (9)']);
 setLogs(prev => [{
 title: 'Успех: Вершина C захвачена',
 description: 'Ребро C-F(9) добавлено в кучу.',
 type: 'success'
 }, ...prev]));
},

```

```

.....
 description: 'Ребро F-I(13) добавлено в кучу. Р
 type: 'success'
 }, ...prev]);
},
// Step 14: Pop D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]);
},
// Step 15: Include D-G, add G's edges
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 eap' } : e));
 setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
 setLogs(prev => [{
 title: 'Успех: Вершина G захвачена',
 description: 'В кучу добавлено G-H(10).',
 type: 'success'
 }, ...prev]);
},
// Step 16: Pop C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
 description: 'Вершины C и F уже заражены плесенью',
 type: 'warning'
 }, ...prev]);
},
// Step 17: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)']

```

```

.....
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setHeapQueue(['H-I (12)', 'F-I (13)']));
 setLogs(prev => [{
 title: 'Отказ: Игнорируем E-H',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]);
 },
// Step 22: Pop H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 12: Куча выдает H-I (вес 12)',
 description: 'Плесень тянется к последней чистой',
 type: 'info'
 }, ...prev]);
},
// Step 23: Include H-I
() => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setHeapQueue(['F-I (13 - цикл)']));
 setLogs(prev => [{
 title: ' Успех: Плесень захватила весь граф',
 description: 'Все 9 вершин поглощены. Итоговый г
 иной – мы разрастались из одной точки с помо
 type: 'success'
 }, ...prev]);
},
},
};

// --- BORUVKA STEPS ---
const boruvkaSteps = [
 // Step 1: Five-Year Plan 1 (Inspecting outgoing)
 () => {
 // Highlight the cheapest outgoing edges for EACH
 // Node 0(A) -> A-B(2)
 // Node 1(B) -> A-B(2)

```

```

.....
 оз {D, E, F, G, H, I}.'',
 type: 'success'
 }, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between two villages)
() => {
 // Now both kolkhoses look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мост',
 description: 'Теперь Красный и Синий колхозы од
 -D с весом 5. Остальные мосты B-E(6) и C-F(9)
 type: 'warning'
 }, ...prev]);
},
// Step 4: Concurrently merge into single Kolkhos
() => {
 setVertices(prev => prev.map(v => ({ ...v, color:
 setEdges(prev => prev.map(e =>
 e.status === 'included' || e.id === '0-3' ? {
 }));
 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский
 description: 'Оба колхоза объединились по мосту
 type: 'success'
 }, ...prev]);
},
// Step 5: Check remaining edges & mark as rejected
() => {
 setEdges(prev => prev.map(e => e.status === 'pend
 setLogs(prev => [{
 title: ' Успех: MST построен Борувкой за 2 шага
 description: 'Все лишние ребра (B-E, E-H, C-F,
 вес: 51. Благодаря параллельности, мы потрати
 type: 'success'
 }]);
}

```

```

 });
 } else {
 setLogs([
 {
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 д
 чтобы объединиться в колхоз.',
 type: 'info',
 },
]);
 }
};

```

```

const getColorClass = (color: string, id: number) =>
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 tex
 case 'blue': return 'bg-blue-500 border-blue-300
 case 'purple': return 'bg-purple-600 border-purple
 case 'mold': return 'bg-emerald-500 border-emerald
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIn
 return 'bg-slate-700 border-emerald-500 text-
 }
 return 'bg-slate-700 border-slate-500 text-slate
 }
};

```

```

const getEdgeStroke = (status: string, color: string)
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in_heap') return '#0284c7';
 if (status === 'included') {
 if (color === 'red') return '#ef4444';
 if (color === 'blue') return '#3b82f6';
 if (color === 'purple') return '#a855f7';
 if (color === 'mold') return '#10b981';
 }
 return '#475569';

```



```

.....
 onClick={() => handleReset('prim')}}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'prim'
 ? 'bg-emerald-600 text-white shadow
 : 'text-slate-400 hover:text-slate-1
)
 }
 >
 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>
 <button
 onClick={() => handleReset('boruvka')}}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md
 : 'text-slate-400 hover:text-slate-1
)
 }
 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
 </div>

 <button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-cent
-200 font-semibold text-xs rounded-xl borde
 >
 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
 </button>

 <button
 onClick={handleNextStep}

```

```

.....
 <div className="flex items-center gap-4">
 <div className="w-3 h-3 rounded-full bg-slate-300">
 </div>
 </div>
) : activeAlgo === 'prim' ? (
 <div className="flex items-center gap-4">
 <div className="w-3 h-1 bg-sky-600">
 </div>
 <div className="flex items-center gap-4">
 <div className="w-3 h-3 rounded-full bg-slate-300">
 </div>
 </div>
 </div>
) : (
 <div className="flex items-center gap-4">
 <div className="w-3 h-3 rounded-full bg-slate-300">
 </div>
 <div className="flex items-center gap-4">
 <div className="w-3 h-3 rounded-full bg-slate-300">
 </div>
 <div className="flex items-center gap-4">
 <div className="w-3 h-3 rounded-full bg-slate-300">
 </div>
 </div>
 </div>
 </div>
)}
 </div>

 <div className="absolute top-4 right-4 bg-slate-300 p-4 ext-slate-300">
 {Math.floor(stepIndex / currentSteps.length)}
 </div>

 {/* SVG Edges */}
 <svg className="absolute inset-0 w-full h-full" style={{
 aspectRatio: "none"
 }}>
 {edges.map(edge => {
 const u = vertices[edge.u];

```

```

.....
 { /* HTML Overlay for Vertices */}
 <div className="absolute inset-0 w-full h-full" >
 {vertices.map(vertex => {
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: vertex.x + "%",
 className={"absolute -translate-x-1/2 -translate-y-1/2"},
 font-weight: 'bold', text-align: 'center', border: '2px solid black', transition: 'all 0.2s ease-in-out' }
 id={vertex.id}}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'prims' ? (

 MST построено с помощью {activeAlgo}

) : null}
 </div>
)}
 </div>

 {stepIndex >= currentSteps.length && (
 <div className="absolute inset-x-6 bottom-6 shadow-xl z-20" >
 <p className={"font-weight: bold; text-align: center; font-size: 1.2em; color: #333; padding: 5px 0"} >
 Шаг {stepIndex + 1} из {currentSteps.length}
 </p>
 <p className="text-slate-300 text-xs mt-2" >
 Общий вес минимального остова: 51. Всего ребер: 5
 </p>
 </div>
)}
 </div>

 { /* Logs / Heap */}
 <div className="bg-slate-950 rounded-2xl border-1px border-slate-800 p-4" >
 <div className="p-4 bg-slate-900 border-bottom border-slate-800" >
 <h4 className="font-weight: bold; text-slate-200 text-sm" >
 Ход выполнения
 </h4>

```

```

.....
 {log.type === 'success' && <CheckCircle/>&&
 {log.type === 'warning' && <HelpCircle/>&&
 {log.type === 'error' && <XCircle/>&&
 {log.type === 'info' && <Play/>&& className="text-xl"}
 <h5 className="font-bold text-sm text-slate-300">
 </div>
 <p className="text-xs text-slate-300">
 </div>
 </div>
</div>
</div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-slate-800 p-1 rounded"
 <div className="flex flex-col sm:flex-row items-center gap-2.5">
 <div className="flex items-center gap-2.5">
 <BookOpen className="w-5 h-5 text-indigo-400"/>
 <h4 className="text-xl font-bold text-white">
 </div>

```

```

<div className="flex bg-slate-950 p-1 rounded"
 <button
 onClick={() => setActiveCheatTab('kruskal')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 " " +
 (activeCheatTab === 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300")
 }
 >
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => setActiveCheatTab('prim')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 " " +
 (activeCheatTab === 'prim' ? "bg-emerald-600 text-white" : "bg-slate-800 text-slate-300")
 }
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('dijkstra')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 " " +
 (activeCheatTab === 'dijkstra' ? "bg-teal-600 text-white" : "bg-slate-800 text-slate-300")
 }
 >
 Дейкстра (Билет 20)
 </button>

```



```

.....
 </div>
 </div>
 </div>
 })

 {activeCheatTab === 'boruvka' && {
 <div className="grid grid-cols-1 lg:grid-cols-2" >
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Clock className="w-3.5 h-3.5 text-rose-500" />
 </p>
 <p className="text-2xl font-black text-white">Время
 <p className="text-slate-400 text-xs mt-1">
 п>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-rose-500" />
 </p>
 <p className="text-2xl font-black text-white">Награда
 <p className="text-slate-400 text-xs mt-1">
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 ении:</p>
 <p className="text-xs text-slate-300 leading-5">
 , организовав массовое распараллеленное слияние
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-rose-500" />
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
 80 flex-1">
 {`def boruvka(n, edges):

```

```

.....
 alt: 'Добрый робот Дейкстра',
 title: ' Добрый (Дейкстра)',
 desc: 'Быстрый, жадный, но работает только с',
 highlight: 'положительными',
 highlightColor: 'text-emerald-400',
 rest: 'весами. Подсунь отрицательное ребро – сломает',
 complexity: 'O((V+E) log V)',
 border: 'border-blue-500/30 hover:border-blue-400/6',
 titleColor: 'text-blue-400',
 badge: 'text-blue-400/70 bg-blue-950/40',
},
{
 img: bellmanImg,
 alt: 'Фура по болоту Форд-Беллман',
 title: ' Фура по Болоту (Ф-Б)',
 desc: 'Медленный, тяжёлый – зато тащит',
 highlight: 'отрицательные',
 highlightColor: 'text-rose-400',
 rest: 'веса и детектит отрицательные циклы.',
 complexity: 'O(V · E)',
 border: 'border-rose-500/30 hover:border-rose-400/6',
 titleColor: 'text-rose-400',
 badge: 'text-rose-400/70 bg-rose-950/40',
},
{
 img: floydImg,
 alt: 'Все-ко-Все Флойд',
 title: ' Все ко Все (Флойд)',
 desc: 'Матрица + три цикла (',
 highlight: 'k, i, j',
 highlightColor: 'text-emerald-400',
 rest: ')). Ищет пути от всех ко всем.',
 complexity: 'O(V³)',
 border: 'border-emerald-500/30 hover:border-emerald',
 titleColor: 'text-emerald-400',
 badge: 'text-emerald-400/70 bg-emerald-950/40',
},
};

```

```

return (
 <header className="bg-slate-900 border-b border-slate-800">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 <div className="flex items-center gap-3 w-full">
 <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium">
 Подготовка к экзаменам: Алгоритмы, Структуры данных
 </p>
 </div>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto">
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-white duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-white duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Code className="w-4 h-4" /> Симулятор
 </button>
 </div>
 </div>
)

```



Универсальное пособие • полный исходный код

ФАЙЛ 41/61: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
export function PlanarityDemo() {
 return (
 <div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5
 <p className="text-xs text-slate-400">Просто смотрим
 <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 { /* K5 */ }
 <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0" >
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return null;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 function ccw(ax:number,ay:number,bx:number,by:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 }
 })() }
```



## Универсальное пособие • полный исходный код

```
.....
 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
}

const PRESETS = [
 { name: 'Студент Вася', emoji: '🍷', color: 'from-blue',
 text: 'Порцию матана!' },
 { name: 'Кодер Семён', emoji: '🍲', color: 'from-violet',
 text: 'Порцию борща!' },
 { name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
 text: 'Порцию сметаны, плиз.' },
 { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose',
 text: 'Порцию борща!' },
 { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald',
 text: 'Порцию пива для нашего НЛО!' },
 { name: 'Бизнесмен', emoji: '💼', color: 'from-slate',
 text: 'Порцию тартанов.' },
];

let counter = 3;

export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: '🍷', color: 'from-blue',
 text: 'Порцию матана!', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: '🍲', color: 'from-violet',
 text: 'Порцию борща!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
 text: 'Порцию сметаны, плиз.', animState: 'idle' },
]);

 const [servedHistory, setServedHistory] = useState<string[]>([]);
 const [isServing, setIsServing] = useState(false);
 const [shakeEmpty, setShake] = useState(false);
 const [log, setLog] = useState<string[]>([]);

 const addLog = (msg: string) => setLog(prev => [...prev, msg]);
```



```

.....
 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800
 r-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
 <div className="grid grid-cols-1 md:grid-cols-12" >

 { /* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */ }
 <div className={`md:col-span-9 bg-slate-900 rounded-2xl
 .-[300px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`>
 <div className="absolute top-4 left-4 text-[1.2em] font-mono" >
 Очередь голодных гостей (Буфер FIFO / BFS)
 </div>

 <div className="flex-1 flex flex-col justify-between" >
 <div className="flex items-end justify-between" >

 { /* ПОВАРИХА И КОТЕЛ С СУПОМ */ }
 <div className="flex flex-col items-center" >
 <div className="relative" >
 <div className="absolute -top-10 left-1/2 -translate-x-1/2" >
 <div className="w-1.5 h-6 bg-slate-400 rounded" >
 <div className="w-2 h-4 bg-slate-400 rounded" >
 </div>

 </div>
 <div className="text-center" >
 <

 ПОВАР

 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>

```

```

.....
 <div className="absolute -bottom-2 text-4xl font-400
it rotate-45"></div>
 </div>

 {/* Персонаж */}
 <div className={`
 relative
 w-14 h-14 rounded-2xl border-1px
 ${customer.color}
 ${isHead ? 'ring-4 ring-amber-500' : ''}
 `}>
 {customer.name}

 {/* Индексы HEAD/TAIL */}
 {(isHead || isTail) && (
 <span className={`
 absolute -bottom-2 text-4xl font-400
 ${isHead ? 'bg-amber-500' : 'bg-slate-900'}
 `}>
 {isHead && isTail ? 'h+t' : isHead ? 'h' : 't'}

)}
 </div>
 </div>
 </div>

 {/* Пол кухни */}
 <div className="w-full h-3 bg-gradient-to-r from-amber-500 to-slate-900">
 </div>

 {/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
 <div className="md:col-span-3 bg-slate-900 rounded-2xl p-4">

```

```
.....
 <div className="bg-slate-900 border border-slate-800" data-bbox="171 101 1000 121">
 <div className="text-[10px] font-mono text-slate-400" data-bbox="204 124 1000 144">
 {log.map((entry, idx) => (
 <div data-bbox="237 174 1000 259">
 key={idx}
 className={`text-xs font-mono flex items-center justify-between gap-2`} data-bbox="271 221 1000 241">
 >
 {idx === 0 ?
 {entry}
 </div>
 </div>
 </div>
 </div>
))}
 </div>
 </div>
);
};
```

```
.....
ФАЙЛ 43/61: src/components/SalmonAutomatonWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО
.....
```

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
 id: number;
 char: string;
 parent: number;
 children: { [key: string]: number };
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
 y?: number;
}
```

```

.....
wordList.forEach((word) => {
 let curr = 0;
 for (let i = 0; i < word.length; i++) {
 const c = word[i].toUpperCase();
 if (newNodes[curr].children[c] === undefined) {
 count++;
 newNodes[curr].children[c] = count;
 newNodes[count] = {
 id: count,
 char: c,
 parent: curr,
 children: {},
 fail: 0,
 term_link: 0,
 is_terminal: false,
 words: [],
 depth: newNodes[curr].depth + 1
 };
 }
 curr = newNodes[curr].children[c];
 }
 if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
 }
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 }

```



```

.....
 setWords(updated);
 setNewWord('');
 buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
 if (words.length <= 1) {
 setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем продолжить');
 return;
 }
 const updated = words.filter((w) => w !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const updatedNodes = { ...nodes };
 rootChildren.forEach((childId) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на главном уровне суффикса просто нет. Жми «Следующий шаг»!'
);
};

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 }
};

```

.....

```

 setNodes(updatedNodes);
 setBfsQueue(newQueue);
 setCurrentNode(r);
 setSpeechText(logs);
};

```

```

return (
 <div className="bg-slate-800/90 rounded-2xl p-6 border"
 /* Шапка виджета */
 <div className="flex flex-wrap items-center justify-between"
 <div className="flex items-center space-x-3">

 <div>
 <h4 className="text-2xl font-bold text-white">
 Интерактивный конструктор: Говорящий Эхо-
 </h4>
 <p className="text-sm text-slate-400">
 Построение дерева и расчет суффиксных ссы
 </p>
 </div>
 </div>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate-700
 transition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
 </div>
 </div>

```

```

/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 /* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center justify-center"

```

```

.....
 })

 { /* Рот (анимация разговора) */}
 {isTalking ? (
 <path d="М 165 105 Q 155 120 165 125 Q
) : (
 <path d="М 165 110 Q 155 115 168 118" s
)}

 { /* Улыбка / жабры */}
 <path d="М 130 80 Q 125 100 130 120" stro
 <path d="М 120 85 Q 115 100 120 115" stro
</svg>

 { /* Декоративные пузыри */}
 <div className="absolute -top-1 right-4 tex
 <div className="absolute top-8 right-1 text
</div>
 <span className="mt-3 bg-rose-600/30 border b
 Эхо-Лосось Сеня

</div>

 { /* Пузырь с текстом (Баббл) */}
 <div className="md:col-span-2 bg-slate-800 bord
 tween min-h-[140px]">
 { /* Треугольник баббла */}
 <div className="absolute -left-3 top-1/2 -tran
 slate-800 border-b-[12px] border-b-transpar

 <div className="flex items-center space-x-2 t
 <Volume2 className={`w-4 h-4 ${isTalking ?
 Прямое вещание из аквариума:
 </div>

 <p className="text-slate-200 text-base leading
 {speechText}
 </p>

```

```

 </div>
 </div>
 </div>

 { /* SVG Canvas for Automaton Tree */
 <div className="mb-8 bg-slate-900 border border-slate-800" >
 <h5 className="text-white font-bold mb-3 flex items-center" >
 Дерево и Суффиксные (fail) ссылки
 </h5>

 {(() => {
 let maxX = 0;
 let maxY = 0;
 Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
 });
 const svgWidth = Math.max(maxX + 80, 400);
 const svgHeight = Math.max(maxY + 60, 200);

 return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`} >
 <defs>
 <marker id="arrowhead" markerWidth="6" markerHeight="6" >
 <polygone points="0 0, 6 3, 0 6" fill="white" />
 </marker>
 <marker id="fail-arrow" markerWidth="6" markerHeight="6" >
 <polygone points="0 0, 6 3, 0 6" fill="white" />
 </marker>
 </defs>

 { /* Draw Edges */
 Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x || !node.y) return null;

 const isCurrent = currentNode === node.id
 })
)
)
 })}
 </div>
)
}

```

```

.....
 return (
 <path
 key={`fail-${node.id}`}
 d={`M ${node.x} ${node.y - 10} Q ${
- 16}`}
 fill="none"
 stroke={isCurrent ? '#f43f5e' : 'rgb(0,0,0)'}
 strokeWidth={isCurrent ? 2 : 1.5}
 strokeDasharray="4 4"
 markerEnd="url(#fail-arrow)"
 />
);
 })}

{/* Draw Nodes */}
Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node.id;
 const isInQueue = bfsQueue.includes(node.id);

 let fill = '#1e293b';
 let stroke = '#475569';

 if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
 } else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }

 return (
 <g key={`node-${node.id}`}>
 <circle

```

```
.....
<div>
 <h5 className="text-white font-bold mb-3 flex flex-wrap gap-2">
 Словарь (Паттерны)
 </h5>
 <div className="flex flex-wrap gap-2 mb-4">
 {words.map((w) => (
 <span
 key={w}
 className="bg-slate-800 border border-slate-700 border-radius-4 px-2 py-1 group"
 >
 {w}
 <button
 onClick={() => removeWord(w)}
 className="text-slate-500 hover:text-white"
 title="Удалить"
 >
 x
 </button>

))}
 </div>
</div>
```

```
<form onSubmit={handleAddWord} className="flex flex-wrap gap-2">
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.value)}
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-slate-700 border-radius-4 px-2 py-1 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-600 text-white font-medium py-2 px-4 transition-colors"
 >
 Добавить
 </button>
</form>
```



```
};
```

```
.....
ФАЙЛ 44/61: src/components/SegmentTreeVisualizer.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО
```

```
.....
import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
```

```
// ===== TYPES =====
```

```
interface Step {
 id: number;
 desc: string;
 codeLine: number;
 treeState: Record<number, number>; // node index
 highlightNodes: number[]; // current highlighted nodes
 mergeChildren?: [number, number]; // pair being merged
 arrayHighlight?: [number, number]; // [L, R] range
 result?: number; // partial result
}
```

```
// ===== PURE LOGIC: build tree =====
```

```
function buildTreeFull(arr: number[]): Record<number, number> {
 const n = arr.length;
 const tree: Record<number, number> = {};
 function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 }
 go(1, 0, n - 1);
 return tree;
}
```



## Универсальное пособие • полный исходный код

```
.....
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 steps.push({ id: id++, desc: `Объединяем: tree[${v}]
 state: { ...tree }, highlightNodes: [v], mergeChildren:
 });
 }
 go(1, 0, n - 1);
 steps.push({ id: id++, desc: `Дерево построено! Корень:
 Highlight: [0, n - 1] });
 return steps;
}
```

### // 2. Top-Down Query steps

```
function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];
 let id = 0;

 function go(v: number, l: number, r: number, ql: number, qr: number) {
 if (ql > r || qr < l) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 2, treeState: tree, highlightNodes: [v],
 });
 return 0;
 }
 if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 value: arr[v], codeLine: 4, treeState: tree, highlightNodes: [v],
 });
 return tree[v];
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 6, treeState: tree, highlightNodes: [v],
 });
 const left = go(2 * v, l, m, ql, qr);
 const right = go(2 * v + 1, m + 1, r, ql, qr);
 const res = left + right;
 steps.push({ id: id++, desc: `Возврат в узел ${v}: ${res}`,
 mergeChildren: [2 * v, 2 * v + 1], arrayHighlight: [v],
 });
 return res;
 }
}
```

```

 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = ${l}, r = ${r}`,
 , arrayHighlight: [qL, qR], result: res });
 }
}
steps.push({ id: id++, desc: `Запрос sum(${qL}..${qR})`,
 ighlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; }
```

```
function buildVisualTree(treeState: Record<number, number>) {
 if (l === r) return { v, l, r, val: treeState[v] ?? null };
 const m = (l + r) >> 1;
 return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(
 treeState, v + 1, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```
const BUILD_CODE = [
 "build(v, l, r) {",
 " if (l === r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 " tree[v] = tree[2*v] + tree[2*v+1];",
 "}"
]

```

```
};
```

```
const QUERY_TD_CODE = [
 "query(v, l, r, ql, qr) {",
 " if (ql > r || qr < l) return 0;",

```

## Универсальное пособие • полный исходный код

```
.....
const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-
border-indigo-500 bg-indigo-600', childRing: 'ring-
-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e
30 border-emerald-500 bg-emerald-600', childRing: '
-slate-900' }, amber: { btn: 'bg-amber-600 hover:bg
der-amber-500 bg-amber-600', childRing: 'ring-rose-
[color];

return { step, idx, setIdx, playing, setPlaying, speed
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr
| 'emerald' | 'amber'; arr: number[] }) {
 const p = Player({ steps, color });
 if (!p) return null;
 const { step, idx, setIdx, playing, setPlaying, speed
 const n = arr.length;

 const vTree = useMemo(() => buildVisualTree(step.tree)

 const renderNode = useCallback((nd: VNode): React.Rea
 const isHigh = step.highlightNodes.includes(nd.v);
 const isChild = step.mergeChildren?.includes(nd.v);
 const has = nd.val !== null;

 let cls = 'bg-slate-800 border-slate-700 text-slate
 if (isHigh) cls = `${clr.ring} text-white ring-2 sc
 else if (isChild) cls = `${clr.childRing} text-ambe
 else if (has) cls = `${clr.filled} text-slate-200`;

 return (
 <div key={nd.v} className="flex flex-col items-ce
 <div className={`flex flex-col items-center jus
 ${cls}`}>
 <span className="text-[7px] opacity-60 font-m
 [{nd.l
 <span className="text-xs font-extrabold font-
```

```

 { /* Tree */
 <div className="flex-1 p-2 overflow-x-auto bg-slate-900">
 <div className="scale-[0.85] origin-top min-w-max">
 </div>

 { /* Code */
 <pre className="px-3 py-2 text-[9px] md:text-[10px] font-mono">
 x">
 {code.map((line, i) => {
 const ln = i + 1;
 const active = step.codeLine === ln;
 return (
 <div key={ln} className={`px-2 py-0.5 rounded-
 2 border-indigo-500 text-indigo-200' : color
 ' : 'bg-amber-600/20 border-l-2 border-amber-

 </div>
);
 })}
 </pre>

 { /* Description + result */
 <div className="px-3 py-2.5 bg-slate-900/60 border-t-1 border-slate-800">
 <span className={`mt-0.5 inline-block w-2 h-2 rounded-
 2 border-indigo-500 text-indigo-200' : color
 ' : 'bg-amber-600/20 border-l-2 border-amber-
 {step.desc}
 {step.result !== undefined && (
 <span className={`ml-auto shrink-0 font-mono text-
 ' : color === 'emerald' ? 'text-emerald-300' :
 = {step.result}

)}
 </div>

 { /* Controls */
 <div className="px-3 py-2.5 bg-slate-950 border-t-1 border-slate-800">
 <div className="flex items-center bg-slate-900">

```



```

</div>
<div className="flex items-center gap-1">

 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 />
 </div>
</div>
<div className="text-[10px] text-slate-500">
 Запрос: $\text{sum}(A[\{qL\}..\{qR\}]) = \{arr.slice(qL, qR+1).reduce((acc, val) => acc + val, 0)}\}$
</div>
</div>
</div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1">
 <button onClick={() => setView('build')} className="text-sm font-medium transition-colors duration-200" >
 Построить
 </button>
 <button onClick={() => setView('queries')} className="text-sm font-medium transition-colors duration-200" >
 Запрос: св
 </button>
 <button onClick={() => setView('theory')} className="text-sm font-medium transition-colors duration-200" >
 Почему / Спра
 </button>

```

```

{view === 'theory' && {
 <div className="space-y-5">
 {/* Why it splits this way */}
 <div className="bg-slate-950 p-5 rounded-xl b
 <h4 className="text-base font-bold text-whi
 во разбилось именно так для N={arr.length}?
 <p className="text-sm text-slate-300 leadin
 Каждый узел берёт <code className="bg-sla
 к получает <code className="bg-slate-900 px
 -slate-900 px-1 rounded font-mono text-emerg
 евая часть забирает на 1 больше (округление
 </p>
 <div className="bg-slate-900 p-4 rounded-lg
 <div className="text-slate-400">Корень: L
 <div className="text-slate-300">mid = {(0
 <div className="text-indigo-300">→ Левый:
 <div className="text-emerald-300">→ Правы
 h - 1) >> 1)} эл.)</div>
 <div className="text-slate-500 mt-2">Всег
 } внутренних.</div>
 </div>
 </div>

 {/* Comparison cards */}
 <div className="grid grid-cols-1 xl:grid-cols
 <div className="bg-slate-950 p-5 rounded-xl
 <div className="absolute -top-3 left-4 bg
 e border border-emerald-500 shadow-md">
 Запрос Сверху-вниз (Рекурсия)
 </div>
 <p className="text-xs text-slate-300 mb-3
 Начинаем с корня. Если отрезок узла полн
 наче спускаемся в обоих детей.
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between bo
 span className="text-rose-400 font-mono">0(
 <div className="flex justify-between bo

```

```
}
.....
```

```
.....
ФАЙЛ 45/61: src/components/Sidebar.tsx
```

```
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО
```

```
.....
import React from 'react';
import { chapters, activeVizIds } from '../data/content';
import { ChevronRight, Compass } from 'lucide-react';
```

```
interface SidebarProps {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
}
```

```
export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId }) => {
 return (
 <aside className="w-full lg:w-80 bg-slate-900/80 border-r border-slate-700">
 <div>
 <h3 className="text-xs font-bold text-slate-400">
 <Compass className="w-4 h-4 text-indigo-400" />
 </h3>
 <div className="space-y-2">
 {chapters.map((chapter) => {
 const isSelected = chapter.id === selectedChapterId;
 const hasViz = activeVizIds.includes(chapter.id);
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id);
 setTimeout(() => {
 const el = document.getElementById(chapter.id);
 if (el && window.innerWidth < 1024) {
 // On mobile we scroll just past the button
 // scrollIntoView is safer on mobile
 }
 }, 100);
 }}
 >
 {chapter.title}
 </button>
);
 })}
 </div>
 </div>
 </aside>
);
}
```



```
.....
 <p className="leading-relaxed">
 Каждая глава разделена на две колонки: слева
 ь.
 </p>
 <p className="text-slate-400 font-medium italic">
 Душные детали, формулы и код спрятаны в спойлере.
 </p>
 </div>
</aside>
);
};
```

.....  
ФАЙЛ 46/61: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, ArrowLeft, ArrowUp, ArrowDown } from 'lucide-react';
```

```
interface Plate {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Person {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Hypothesis {
 id: string;
 text: string;
 probability: number;
}
```

```

 };
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(` Положили поверх стопки: ${newPlate.name} `);
 };

 const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(` Вымыта верхняя тарелка (LIFO): ${topPlate.name} `);
 };

 const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
 { id: '3', name: 'Блюдец от торта', icon: '🍰' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
 };

 // Queue handlers
 const addPerson = () => {
 const presets = [
 { name: 'Любитель борща', icon: '🥘' },
 { name: 'Курьер доставки', icon: '📦' },
 { name: 'Усталый админ', icon: '🧑‍💻' },
 { name: 'Турист с рюкзаком', icon: '🎒' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue(prev => [...prev, newPerson]);
 setAddMessage(` Добавили в очередь: ${newPerson.name} `);
 };

 const removePerson = () => {
 if (queue.length === 0) {
 setRemoveMessage("🚫 Очередь пуста!");
 return;
 }
 const frontPerson = queue[0];
 setQueue(prev => prev.slice(1));
 setRemoveMessage(` Удален из очереди: ${frontPerson.name} `);
 };

 const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Любитель борща', icon: '🥘' },
 { id: '2', name: 'Курьер доставки', icon: '📦' },
 { id: '3', name: 'Усталый админ', icon: '🧑‍💻' },
 { id: '4', name: 'Турист с рюкзаком', icon: '🎒' },
]);
 setRemoveMessage(" Очередь восстановлена.");
 };

```

```

.....
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(` Добавлена гипотеза с вероятностью
};

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability)
 maxIdx = i;
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(` Выбран самый "тяжелый" кандидат:
};

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"' },
]);
 setHeapMessage(" Гипотезы восстановлены.");
};

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

return (
 <div className="bg-slate-900/90 border border-slate-800 pb-6 mb-6">
 <div className="border-b border-slate-800 pb-6 mb-6">
 <h2 className="text-2xl font-bold text-transparent bg-clip-text">

```

```

.....
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab === 'fifo' ? 'bg-indigo-500' : 'bg-slate-800'} rounded-sm`} />
 <div className="text-left">
 <div className="font-bold text-sm text-white">FIFO
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>
 BFS

</button>

<button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between gap-2 p-2
 ${activeTab === 'heap' ? 'bg-emerald-600/20 border-emerald-500 text-white' : 'bg-slate-800/60 border-slate-700/60 text-white'}
 `}
>
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'bg-emerald-500' : 'bg-slate-800'} rounded-sm`} />
 <div className="text-left">
 <div className="font-bold text-sm text-white">Приоритетная
 <div className="text-xs opacity-80">Приоритетная
 </div>
 </div>
 Beam Search

</button>
</div>

{/* STACK TAB */}
{activeTab === 'stack' && (
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-xl">
 <div className="font-bold text-sm text-white">Stack
 <div className="text-xs opacity-80">Stack
 </div>
 </div>
)}

```

```

.....

 {popMessage}
 </div>
)}

 { /* Visualization */ }
 <div className="bg-slate-950/60 p-6 rounded-2xl" style={color} >
 >
 {stack.length === 0 ? (
 <div className="text-center py-12 text-slate-400" >
 <div className="text-5xl mb-3" > </div>
 <p className="text-base font-bold">Стек пуст</p>
 <p className="text-xs mt-1">Добавьте элемент</p>
 </div>
) : (
 <div className="w-full max-w-sm flex flex-direction-column" >
 <div className="absolute top-0 text-xs font-medium text-slate-400" >
 full flex items-center gap-1" >
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 animate-pulse" />
 </div>

 { /* Reversed map so top of stack is visible */ }
 {stack.slice().reverse().map(({plate, index}) => {
 const isTop = index === 0;
 return (
 <div
 key={plate.id}
 className={`w-full flex items-center justify-between p-2
 ${isTop
 ? 'bg-gradient-to-r from-blue-500 to-blue-600'
 : 'bg-slate-900/90 border-slate-800'}
 `} >
 <div>
 {plate.id}
 {plate.name}
 </div>
 <div>
 {plate.amount}
 {plate.isFull}
 </div>
 </div>
)
 })}
 </div>
 </div>
)}
 </div>
 <div className="flex items-center justify-between p-4" >
 {plate.name}
 {plate.amount}
 </div>

```

```

.....
 className="flex-1 md:flex-none flex item
-2.5 rounded-xl text-sm font-bold shadow-lg
 >
 <Plus className="w-4 h-4" /> Вставить в о
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex item
er border-indigo-500/40 px-4 py-2.5 rounded
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
</div>

{dequeueMessage && {
 <div className="bg-indigo-950/60 border bord
p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {dequeueMessage}
 </div>
)}

/* Visualization */
<div className="bg-slate-950/60 p-6 rounded-2
overflow-x-auto">
 {queue.length === 0 ? {
 <div className="text-center py-12 text-sla
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Очеп

```

```

 }`}>
 {isFirst ? 'Получает сун' : 'В

 </div>
}
});
}}}

 <div className="flex flex-col items-center
-600 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />
 <span className="text-xs font-mono up
 </div>
 </div>
})
<div className="mt-6 text-xs text-slate-500
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
</div>
</div>
</div>
})

{/* HEAP TAB */}
{activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Приоритетная очередь / Куча</sp
 <span className="text-xs font-mono bg-e
 x-Heap
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Здесь не важно, кто пришел первым. Важн
 </p>
 </div>
 <div className="flex items-center gap-3 w-f
 <button

```

```

 max="0.99"
 step="0.01"
 value={newCustomProb}
 onChange={e => setNewCustomProb(Number(e.target.value))}
 className="w-full accent-emerald-500 cursor-pointer"
 />
 </div>
 <div className="md:col-span-3">
 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center justify-center text-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
 >
 <Plus className="w-4 h-4" /> Добавить в список
 </button>
 </div>
 </form>

 {heapMessage && (
 <div className="bg-emerald-950/60 border border-emerald-500/40 p-3 animate-fade-in">

 {heapMessage}
 </div>
)}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2xl">
 <div>
 {sortedHeap.length === 0 ? (
 <div className="text-center py-12 text-slate-400">
 <div className="text-5xl mb-3">0</div>
 <p className="text-base font-bold">Куча пуста</p>
 <p className="text-xs mt-1">Добавьте элемент</p>
 </div>
) : (

```



```

<div className="flex items-center
 /* Custom progress bar */
 <div className="hidden sm:block
 <div
 className={`h-full rounded-
 style={{ width: `${percent}%
 ></div>
 </div>
 <span className={`font-mono font
 isTop ? 'text-emerald-400 tex
 }`}>
 {percent}%

 </div>
</div>
);
)})
</div>
})
<div className="mt-6 text-xs text-slate-500
 НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
</div>
</div>
</div>
 })
</div>
);
};

```

ФАЙЛ 47/61: src/components/SplayTreeViz.tsx

**КАТЕГОРИЯ:** Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from "react";
import {
 Play,

```

```

.....
 x.right = y.left;
 y.left = x;
 return y;
};

// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
 if (!node) return null;
 return {
 key: node.key,
 left: cloneTree(node.left),
 right: cloneTree(node.right),
 x: node.x,
 y: node.y,
 };
};

export const SplayTreeViz: React.FC = () => {
 // Let's create an elegant initial unbalanced or semi-balanced tree
 // Values: 10, 20, 30, 40, 50, 60
 const createInitialTree = (): SplayNode => {
 let r: SplayNode | null = null;
 const initialKeys = [40, 20, 50, 10, 30, 60];
 initialKeys.forEach((key) => {
 r = insertBST(r, key);
 });
 return r!;
 };

 const [tree, setTree] = useState<SplayNode>(createInitialTree());
 const [inputValue, setInputValue] = useState<string>('');
 const [log, setLog] = useState<string[]>([
 'Дерево инициализировано. Попробуйте кликнуть на узел',
]);
 const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

 // Splay operation that tracks steps!
 const splayStepByStep = (

```

```

 });
}

// Now let's implement standard Splay logic recursively
const splayAction = (
 node: SplayNode | null,
 k: number,
): SplayNode | null => {
 if (!node || node.key === k) return node;

 if (k < node.key) {
 if (!node.left) return node;

 if (k < node.left.key) {
 // Zig-Zig (Left-Left)
 node.left.left = splayAction(node.left.left, k);
 node = rightRotate(node);
 steps.push(
 `Вращение Zig-Zig (Правый поворот родителя)`
);
 } else if (k > node.left.key) {
 // Zig-Zag (Left-Right)
 node.left.right = splayAction(node.left.right, k);
 if (node.left.right) {
 node.left = leftRotate(node.left);
 steps.push(`Компонент Zig-Zag: левый поворот`);
 }
 }
 }

 if (!node.left) return node;
 steps.push(
 `Финальный поворот Zig (Правое вращение корня)`
);
 return rightRotate(node);
} else {
 if (!node.right) return node;

 if (k < node.right.key) {

```

```

.....
 const val = parseInt(inputValue);
 if (isNaN(val)) return;

 // Standard BST insert, then splay!
 let newTree = cloneTree(tree);
 newTree = insertBST(newTree, val);
 const { newRoot, steps } = splayStepByStep(newTree,
 if (newRoot) {
 setTree(newRoot);
 }
 setLog(['Вставили вершину ${val}'].`, ...steps]);
 setInputValue("");
 setHighlightedNode(val);
};

const handleFind = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;
 handleSplay(val);
 setInputValue("");
};

const resetTree = () => {
 setTree(createInitialTree());
 setHighlightedNode(null);
 setLog(["Дерево возвращено в исходное состояние."]);
};

// Assign coordinate positions using a simple recursive
const computeCoordinates = {
 node: SplayNode | null,
 x: number,
 y: number,
 levelWidth: number,
}: void => {
 if (!node) return;
 node.x = x;

```

```

.....
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.right));
 }
 return lines;
};

const renderNodes = (node: SplayNode | null): React.R
 if (!node) return [];
 let circles: React.ReactNode[] = [];
 const isHighlighted = highlightedNode === node.key;

 circles.push(
 <g
 key={`n-${node.key}`}
 className="cursor-pointer group"
 onClick={() => handleSplay(node.key)}
 title={`Кликните, чтобы выполнить Splay(${node.
 >
 <circle
 cx={node.x}
 cy={node.y}
 r="18"
 fill={isHighlighted ? "#4f46e5" : "#1e293b"}
 stroke={isHighlighted ? "#818cf8" : "#64748b"}
 strokeWidth={isHighlighted ? "3" : "2"}
 className="transition-all duration-300 group-
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="12px"
 fontWeight="bold"

```

```

<form onSubmit={handleFind} className="flex
 <input
 type="number"
 placeholder="Поиск..."
 value={inputValue}
 onChange={(e) => setInputValue(e.target
 className="w-24 bg-slate-950 text-white
00 focus:outline-none"
 />
 <button
 type="submit"
 onClick={handleFind}
 className="bg-indigo-600 hover:bg-indigo
 >
 Поиск / Splay
 </button>
 <button
 type="button"
 onClick={(e) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (!isNaN(val)) {
 let newTree = cloneTree(tree);
 newTree = insertBST(newTree, val);
 const { newRoot, steps } = splaySte
 if (newRoot) setTree(newRoot);
 setLog(`${Вставили ${val}!`, ...st
 setInputValue("");
 setHighlightedNode(val);
 }
 }}
 className="bg-indigo-950 hover:bg-slate
0"
 >
 Вставить
 </button>
</form>

```

```

 </p>
 <p>

 колено. Вращаем узел в разные стороны.

 </p>
 </div>
</div>
</div>

{/* Answers Section inside the visualizer widget */}
<div className="bg-slate-950/80 p-5 border-t border-slate-800">
 <h4 className="text-indigo-400 font-bold text-sm">
 <HelpCircle className="w-4 h-4 text-indigo-400"/>
 Разбор экзаменационных вопросов (Без нытья):
 </h4>

 <div className="grid grid-cols-1 md:grid-cols-3">
 <div className="bg-slate-900/60 p-4 rounded-lg">
 <div>
 <p className="font-bold text-slate-300 mb-2">
 1. Отсутствующий элемент
 </p>
 <p className="text-slate-400 leading-relaxed">
 Допустим, мы ищем{" "}
 <code className="text-indigo-400 font-sans">{1}</code>
 его нет. Алгоритм спустится к листу, на котором
 неудачей (например, {" "})
 <code className="text-indigo-400">{20}</code>
 <code className="text-indigo-400">{30}</code>
 поднимет в корень{" "}
 <strong className="text-white">
 последний успешно просмотренный узел

 , на котором он споткнулся! Это оставляем
 сверху.
 </p>
 </div>
 <div className="mt-3 text-[10px] text-indigo-400">

```

```

.....
 операций. Поэтому амортизированное время
 <strong className="text-white"> $O(\log n)$
 </p>
 </div>
 <div className="mt-3 text-[10px] text-emerald-500">
 Каждое "растяжение" компенсируется "сжатием"
 </div>
 </div>
</div>
</div>
);
};

```

ФАЙЛ 48/61: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';

interface Plate {
 id: string;
 name: string;
 emoji: string;
 color: string;
 isDirty: boolean;
 animState: 'in' | 'idle' | 'washing' | 'clean';
}

const PLATE_PRESETS = [
 { name: 'Тарелка из-под спагетти', emoji: '🍝', color: 'from-pink-500 to-pink-300' },
 { name: 'Тарелка из-под пиццы', emoji: '🍕', color: 'from-pink-500 to-pink-300' },
 { name: 'Блюдец от торта', emoji: '🍰', color: 'from-pink-500 to-pink-300' },
 { name: 'Тарелка от тако', emoji: '🌮', color: 'from-pink-500 to-pink-300' },
 { name: 'Тарелка из-под суши', emoji: '🍣', color: 'from-pink-500 to-pink-300' },

```



```

.....
 ...preset,
 isDirty: true,
 animState: 'in',
 });

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху с

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id !== newP
 }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsWashing(true);
 const topPlate = dirtyStack[dirtyStack.length - 1];

 addLog(` POP: Начинаем мыть верхнюю тарелку с ${top

// Step 1: Start washing animation (sponge and soap
setDirtyStack(prev => prev.map(p => p.id !== topPla
setShowSoap(true);

setTimeout(() => {
 // Step 2: Wash complete. Plate becomes clean and
 const cleanPlate: Plate = {
 ...topPlate,
 isDirty: false,
 animState: 'clean',

```

```
.....
 Попробуешь вытащить нижнюю – всё рухнет!
 </p>
</div>
</div>

{/* УПРАВЛЕНИЕ */}
<div className="flex items-center gap-3 flex-wrap"
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r
 from-slate-800 to-slate-700 opacity-40 disabled:cursor-not-allowed text-white
 font-weight-bold rounded-2xl px-5 py-3.5 transition-all cursor-pointer flex items-center"
 >
 Положить грязную (PUSH)
 </button>

 <button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r
 from-slate-800 to-slate-700 opacity-40 disabled:cursor-not-allowed text-white
 font-weight-bold rounded-2xl px-5 py-3.5 transition-all cursor-pointer flex items-center"
 >
 Помыть верхнюю (POP)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-weight-bold
 cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12">
```

```

<div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}`
>
 <span className="text-xs font-mono text-
er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-
</div>

/* Тарелки (отрисовываем в обратном поряд
{dirtyStack.slice().reverse().map((plate,
 const idx = dirtyStack.length - 1 - rev
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`
 w-full max-w-[280px] h-10 rounded
shadow-md select-none transition-all duration
 ${plate.color}
 ${plate.animState === 'in' ? 'anim
 ${plate.animState === 'washing' ?
 ${isTop ? 'ring-4 ring-blue-500/50
 `}
 >
 {plate.em
 <span className="text-slate-800 tex
 {isTop && (
 <span className="text-[9px] bg-bl
 LIFO

 </div>
)}
 </div>
);
}}}
```

```

 </div>
)}
 </div>

 {/* Столешница сушилки */}
 <div className="w-full h-4 bg-gradient-to-r from-blue-500 to-purple-500">
 </div>
 </div>

 {/* ЛОГ ДЕЙСТВИЙ */}
 <div className="bg-slate-900 border border-slate-700 p-4">
 <div className="text-[10px] font-mono text-slate-400">
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between p-2`}
 >
 {idx === 0 ? История действий : }
 {entry}
 </div>
)}}
 </div>
 </div>
);
};

```

ФАЙЛ 49/61: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useRef, useMemo } from "react";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";

```

```

function generateKmpSteps(pattern: string, text: string) {
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;

```

```

.....
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.` });

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$
 i` });

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` });
 z[i] = Math.min(r - i + 1, z[i - 1]);
 steps.push({ i, l, r, z: [...z], desc: `Пре
 }. Копипаста сработала.` });
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i
 while (i + z[i] < n && s[z[i]] === s[i + z[i]])
 steps.push({ i, l, r, zTarget: z[i], zSource
 падают! Окно растёт.` });
 z[i]++;
 }

 if (i + z[i] < n) {
 steps.push({ i, l, r, zTarget: z[i], zSource
 [i]}` разные. Строч.` });
 } else {
 steps.push({ i, l, r, z: [...z], desc: `Упё
 }

 steps.push({ i, l, r, z: [...z], desc: `Фиксиру

 if (i + z[i] - 1 > r) {
 l = i;

```

```

.....
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
}, {autoPlay, steps.length});

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const step = steps[stepIdx] || steps[0];

return (
 <div className="space-y-4">
 <div className="flex items-center justify-b
 <div className="flex bg-slate-900 borde
 <button onClick={() => setMode("kmp
 className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
 КМП (π-ФУНКЦИЯ)
 </button>
 <button onClick={() => setMode("z")
 className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
 Z-ФУНКЦИЯ
 </button>
 </div>

 <div className="flex items-center gap-2
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te
 <input value={pattern} onChange=
xt-white text-xs font-bold font-mono px-2 p

```

```
.....
-900/40");
```

```
 }
```

```
 }
```

```
 return (
```

```
 <div key={index} className=
```

```
 {/* L/R markers for Z *
```

```
 {mode === "z" && step.l
```

```
 <div className="abs
```

```
)}
```

```
 {mode === "z" && step.r
```

```
 <div className="abs
```

```
)}
```

```
 {/* Index */}
```

```
 <span className={`text-
```

```
 {/* Char Box */}
```

```
 <div className={`w-8 h-
```

```
xtColor} font-mono text-lg transition-color
```

```
 {char}
```

```
 </div>
```

```
 {/* Value array box */}
```

```
 <div className="text-[1
```

```
slate-700/50 font-mono font-bold">
```

```
 {mode === "kmp" ? s
```

```
 </div>
```

```
 {/* KMP fingers */}
```

```
 {mode === "kmp" && step
```

```
 <div className="abs
```

```
z-20">
```

```
 <span classNam
```

```
 </div>
```

```
)}
```

```
 {mode === "kmp" && step
```

```
 <div className="abs
```

```

)
 }}}}
 </div>
 </div>

 <div className="flex flex-col sm:flex-row gap-4" style={{width: '100%'}}>
 { /* Controls */ }
 <div className="flex bg-slate-900 border border-slate-800 rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{width: '100%'}}>
 <button onClick={prevStep} disabled={steps.length === 0} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{width: '50%'}}>
 <Undo strokeWidth={3} size={16} />
 </button>
 <button onClick={playPause} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{width: '50%'}}>
 {autoPlay ? <><Pause size={16} /> : <><Play size={16} />}
 </button>
 <button onClick={nextStep} disabled={steps.length === steps.length} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{width: '50%'}}>
 <SkipForward strokeWidth={3} size={16} />
 </button>
 <button onClick={reset} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{width: '50%'}}>
 <RefreshCw strokeWidth={3} size={16} />
 </button>
 </div>

 { /* Description Log */ }
 <div className="flex-1 bg-slate-900/50 border border-slate-800 overflow-hidden" style={{width: '100%'}}>
 <div className="absolute top-0 right-0 text-white font-mono text-sm" style={{width: '100%'}}>
 {steps.length} / {steps.length}
 </div>
 <div className="text-[10px] text-slate-400" style={{width: '100%'}}>
 {steps.length}
 </div>
 <div className="text-sm text-slate-400" style={{width: '100%'}}>
 {step.desc}
 </div>
 </div>
 </div>

```



```
.....
 </div>
 </div>
);
 }
}
```

```
.....
ФАЙЛ 50/61: src/components/TopologicalSortViz.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО
```

```
.....
import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 ArrowRight,
 BookOpen,
 Clock,
} from "lucide-react";

interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
 name: string;
}

interface TEdge {
 u: number;
 v: number;
}

const dagNodes: TNode[] = [
 { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
 { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
 { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
];
```

```

.....
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
};

recordStep(
 "Начало топологической сортировки. Используем кла
 null,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} ("${dagNodes[u].name}
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 } else if (!fullyProcessed.has(to)) {
 // Explaining potential cycle detected (not in
 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} от
 u,
);
 }
 }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write in

```

```

.....
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Clock className="w-5 h-5 text-indigo-400" />
 Топологическая сортировка (Зависимости обучен
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />

```

```

.....
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
 </marker>
</defs>

{/* Edges */}
{dagEdges.map((edge, idx) => {
 const u = dagNodes.find((n) => n.id === e
 const v = dagNodes.find((n) => n.id === e

 const isActive =
 (step.currentNode === edge.u || step.cu
 step.visited.has(edge.v);

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#334155"}
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-ar
 className="transition-all duration-300
 />
);
}})

{/* Nodes */}
{dagNodes.map((node) => {
 const isCurrent = step.currentNode === no
 const isVisited = step.visited.has(node.id
 const isProcessed = step.fullyProcessed.h

 let fill = "#1e293b";

```

```

>
 ({node.label}) {node.name.split(" ")
</text>
<text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
>
 {node.name.split(" ").slice(1).join
</text>
</g>
);
 }}}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5
 00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

 <div className="bg-slate-900 border border-slate-800">
 <p className="text-xs text-indigo-300 min-h-[100px]">
 {step.desc}
 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-auto">
 {/* Topological List output */}
 <div>

 РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):


```



```

.....
};

export const WaterfallAnimationWidget: React.FC = () => {
 // Переключатель режимов: 'training' (Полный BFS обход)
 const [activeMode, setActiveMode] = useState<'training' | 'search'>('training');

 // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
 const [textToScan, setTextToScan] = useState<string>('');
 const [currentIndex, setCurrentIndex] = useState<number>(0);
 const [currentNode, setCurrentNode] = useState<number>(0);
 const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
 const [searchLog, setSearchLog] = useState<string>('');
 'Режим поиска: Введите текст и жмите «Следующий шаг»';
 };
 const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>({ from: 0, to: 0 });
 const [foundMatches, setFoundMatches] = useState<{ index: number; match: string }[]>([]);
 const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0);

 // === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
 // Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
 const [trainStep, setTrainStep] = useState<number>(0);

 const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
 const clean = e.target.value.toUpperCase().replace(/[^a-zA-Z0-9]/g, '');
 setTextToScan(clean);
 resetSearchSimulation();
 };

 const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся на старт');
 setJumpPath(null);
 setFoundMatches([]);
 setActiveSearchCodeLine(1);
 };
};

```





```

},
{
 targetNodeId: 5,
 title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
 desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
 "H" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'I',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 8,
 title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТАТЬ
 ВАЖНЫМ ВОПРОС! Обрабатываем #8 (SH) на D
). Рыба плывет в ROOT и ищет ветку "H". У корня
 суффикс "H"!',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
 е подходят. fail[HER] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'R',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',

```

```
.....
 { /* Шапка и переключатель режимов */ }
 <div className="flex flex-wrap items-center justify-center">
 <div className="flex items-center space-x-3">

 <div>
 <h4 className="text-2xl font-bold text-white">
 Анимация водопада: Полное обучение автомата
 </h4>
 <p className="text-sm text-slate-400">
 Визуализация пошагового построения всех функций
 </p>
 </div>
 </div>

 { /* Переключатель режимов */ }
 <div className="bg-slate-900 p-1.5 rounded-xl border">
 <button
 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p-2
 ${activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'}
 `}
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корня)
 </button>
 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p-2
 ${activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg shadow-blue-500'
 : 'text-slate-400 hover:text-white'}
 `}
 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
 </button>
```

```

.....
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === 0}
 className="flex-1 bg-slate-700 hover:bg
ion-all"
 >
 Назад
 </button>
 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === TRAINING_STEPS_
 className="flex-1 bg-indigo-600 hover:b
ition-all shadow-lg shadow-indigo-900/50"
 >
 {trainStep === TRAINING_STEPS_INFO.leng
 </button>
 </div>
</div>

{/* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
 >
 <div>
 <h5 className="text-white font-bold mb-3
 <span className="flex items-center gap-
 ❌ Код BFS построения fail

 <span className="text-xs bg-indigo-950
 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 <div className="bg-slate-950 p-4 rounded-
 <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-500 text-indigo-300 font
 1. let u = trie[r][char]; // Te
 {currentTrainInfo.codeLine === 1 && <
 туп}
 </div>
 <div className={`p-1.5 rounded flex ite

```

```

</div>
) : (
 /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
 <div className="grid grid-cols-1 lg:grid-cols-3"
 <div className="bg-slate-900/60 p-5 rounded-x
 <div>
 <h5 className="text-white font-bold mb-2"
 » Текст для сканирования
 </h5>
 <p className="text-xs text-slate-400 mb-4"
 Введи текст, чтобы лосось просканировал
 </p>
 <input
 type="text"
 value={textToScan}
 onChange={handleTextChange}
 maxLength={15}
 placeholder="ВВЕДИТЕ ТЕКСТ..."
 className="w-full bg-slate-800 border b
 cus:outline-none focus:border-blue-500"
 />
 </div>

 <div>
 <div className="mb-4 flex flex-wrap gap-1"
 <span className="text-xs text-slate-400"
 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-cent
 idx === currentIndex
 ? 'bg-blue-600 text-white shadow
 : idx < currentIndex
 ? 'bg-emerald-950/60 text-emera
 : 'bg-slate-800 text-slate-500'
 `}
 >
 {char}

```

```

.....
 border-rose-500 text-rose-300 font-bold an
 2. while (curr != root && !trie
 {activeSearchCodeLine === 2 && <span
рыжок по fail!}
 </div>
 <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold` :
 3. curr = trie[curr].get(char)
 {activeSearchCodeLine === 3 && <span
ем по течению}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 fon
 4. if (is_terminal[curr]) report
 {activeSearchCodeLine === 4 && <span
овпадение!}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-300 font-bold m
 <AlertCircle className="w-3.5 h-3.5 tex
 </h5>
 <div className="bg-slate-800 p-3.5 rounde
tems-center">
 {searchLog}
 </div>
</div>
</div>
</div>
}}

```

```

{/* SVG Анимация водопада */}

```

```

<div className="bg-gradient-to-b from-blue-950 vi
-hidden shadow-2xl">
 {/* Водопадные фоновые эффекты */}
 <div className="absolute inset-0 opacity-10 bg-

```

```

 y2={level.y}
 stroke="#334155"
 strokeWidth="1"
 strokeDasharray="4,4"
 opacity="0.4"
 />
 <rect x="25" y={level.y - 12} width="14"
 />
 <text x="32" y={level.y + 4} fill="#94a0b3"
 {level.label}
 </text>
 </g>
)}}

{/* Рендер прямых переходов (потоков воды) */}
Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
 const fromNode = WATERFALL_NODES[Number(fromId)];
 return Object.entries(children).map(([childId, child]) => {
 const toNode = WATERFALL_NODES[toId];

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' ? true : false;
 type === 'normal';

 // Подсветка в режиме обучения (когда ищем путь)
 let isTrainingExamined = activeMode === 'training' ? true : false;
 let isMatchBranch = isTrainingExamined && childId === toId;

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}`}
 {/* Линия течения воды */}
 <line
 x1={fromNode.x}
 y1={fromNode.y}
 x2={toNode.x}
 y2={toNode.y}
 stroke={isHighlighted ? '#3b82f6' : '#334155'}
 strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
 />
 </g>
);
 });
});

```

```

.....
 // Найдем индекс шага, на котором обработать
 const stepIdxForNode = TRAINING_STEPS_INF
 if (activeMode === 'training' && trainStep

 let isHighlighted = activeMode === 'search
e === 'fail';
 let isJustEstablished = activeMode === 't
-ссылку

 // Вычисляем изогнутую кривую для красоты
 const dx = targetNode.x - node.x;
 const dy = targetNode.y - node.y;
 const cx = node.x + dx / 2 + (node.id % 2
 const cy = node.y + dy / 2 - 30;

 return (
 <g key={`fail-${node.id}`}>
 <path
 d={`M ${node.x} ${node.y} Q ${cx} $
 fill="none"
 stroke={isHighlighted ? '#f43f5e' :
 strokeWidth={isHighlighted || isJus
 strokeDasharray="6,6"
 className={isHighlighted || isJustE
 opacity={isHighlighted || isJustEst
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b
 >
 fail[{node.label}] = {targetNode.
 </text>
)}
 </g>
);
}}
}
}

{/* Рендер вершин (бассейнов водопада) */}
{Object.values(WATERFALL_NODES).map((node)

```

```

.....
 {isTargetChild && {
 <g transform={`translate(${node.x},
 <rect x="-35" y="-12" width="70"
 <text x="0" y="2" fontSize="10" f
 ИЩЕМ FAIL
 </text>
 </g>
 }}

 {/* Индикатор словарных слов */}
 {hasWords && {
 <g transform={`translate(${node.x} +
 <circle cx="0" cy="0" r="9" fill=
 <text x="0" y="3" fontSize="10" f
 *
 </text>
 </g>
 }}
 </g>
 });
}}

{/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */}
<g>
 {/* Круг-подложка под рыбу */}
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5
 stroke="#ffffff"
 strokeWidth="2"
 style={{ transition: 'cx 0.8s cubic-bez
 className="shadow-lg"
 />
 {/* Сама рыба */}
 <text
 x={activeFishPosNode.x + 15}

```



```
.....
 }}}
 </div>
 }}
 </div>
}
</div>
};
};
```

.....  
**РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)**  
.....

.....  
ФАЙЛ 52/61: src/components/visualizers/SparseTableViz.t  
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 661 | РАЗ  
.....

```
import React, { useState, useEffect } from "react";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 Pause,
 RotateCcw,
 ChevronRight,
 ChevronLeft,
 Lock,
 ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
```



```

</h3>
<p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i+1][j])$
</p>
</div>
<div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-500"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 >
 <RotateCcw size={20} />
 </button>
</div>
</div>

<div className="overflow-x-auto">
 <div className="inline-block bg-slate-900 p-6 rounded">
 <div className="grid grid-cols-[auto_repeat(4,1fr)] gap-2">
 <div className="text-slate-500 font-mono text-sm">
 i \ j
 </div>
 <div className="text-center font-bold text-sm">
 2^0 (L=1)
 </div>
 </div>
 </div>
</div>

```

```

 currentStep.i === i - (1 << (currentStep.i - 1))
)
 isSrc2 = true; // wait, no. Src2 is not the source
 // Let's re-eval exact source:
 // We are asking if THIS cell [i][j] is the source
 if (currentStep.j - 1 === j) {
 if (currentStep.i === i) isSrc1 = true;
 if (currentStep.i + (1 << (currentStep.j - 1)) === i)
 isSrc2 = true;
 }
 }
 }

 return (
 <div key={j} className="relative flex flex-col items-center justify-center"
 {!isValid ? (
 <div className="w-12 h-12 flex items-center justify-center"
 <div>
 </div>
 </div>
) : (
 <motion.div
 initial={{ opacity: 0, scale: 0.8 }}
 animate={{
 opacity: isVisible ? 1 : 0,
 scale: isVisible ? 1 : 0.8,
 }}
 className={`w-12 h-12 flex items-center justify-center transition-colors duration-200
 ${isActive
 ? "bg-indigo-500 text-white"
 : isSrc1
 ? "bg-emerald-500 text-white"
 : isSrc2
 ? "bg-amber-500 text-white"
 : isVisible
 ? "bg-slate-800 text-white"
 : "bg-transparent text-white"}
 `}
 >
 {st1D[i][j]}
 </motion.div>
)}
 </div>
);
}

```

```

.....
 const c = computeSteps[step - 1];
 const half = 1 << (c.j - 1);
 return (
 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 = min(

 ST[{c.i}][{c.j - 1}]

 ,

 ST[{c.i + half}][{c.j - 1}]

) → min(
 {st1D[

 {st1D[{c.i + half}][{c.j - 1}]

 } = {" "}

 {st1D[{c.i}][{c.j}]}

 </p>
);
))();
) : (
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
)}
</div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {

```

```

<React.Fragment key={step}>
 {idx > 0 && <div className="text-slate
 <div className={`flex flex-col items-c
 <h4 className={`font-bold mb-3 flex
 }>
 Матрица блоков {stepL}x{step
 <span className={`text-[10px] font
rder-indigo-800/40 text-indigo-300' : 'bg-s
 </h4>
 <div className={`grid gap-[2px] p-2
border-indigo-900/40 shadow-[0_0_25px_rgba(
gridTemplateColumns: `repeat(${stepSize}, 1
 {Array.from({length: stepSize * st
 const r = Math.floor(idx / step
 const c = idx % stepSize;

 let highlightClass = isCurr ? '
/50 cursor-pointer' : 'bg-slate-800/50 text
 let zIndex = 'z-0';

 const isActive = !!activeCell &
ll.c + L) && ((r - activeCell.r) % (1 << st

 if (isActive) {
 if (isCurr) {
 highlightClass = `bg-indigo
{locked && locked.r === r && locked.c === c
 zIndex = 'z-10';
 } else {
 const prevL = 1 << (k - 1);
 const isTop = r < activeCel
 const isLeft = c < activeCe

 if (isTop && isLeft) {
 highlightClass = 'bg-ros
md:scale-[1.08]';
 } else if (isTop && !isLeft
 highlightClass = 'bg-blur

```

```

.....
<div className="flex-1 flex flex-col gap-4">
 <div className="bg-slate-900 p-6 rounded-xl border">
 <h3 className="text-lg font-bold text-slate-300">
 {k === 0 ? (
 <div className="text-slate-400 text-sm leading-6">
 <p className="mb-4">При
 ы имеют размер
 <div className="bg-[#0a0ffe] rounded-xl p-4">
 // Ба
 ST
 </div>
 <p className="mt-6 italic text-indigo-400">
 <Play size={14} className="fill-indigo-400">
 </p>
 </div>
) : (
 <div className="flex flex-col gap-4 animate-pulse">
 <div className="bg-[#0a0ffe] rounded-xl p-4 border border-slate-300 shadow-inner overflow-x-auto">
 // Те
 int<
 ="text-amber-300">1);

 // Кв
 ST

 e="text-amber-300">1}, <span

 ext-amber-300">1][k-
 pan>

 ="text-amber-300">1][k-
 span>

 white">c + pL][k-
 ame="text-slate-500">// Правый Низ

 {''})});
 </div>

```

```

.....
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
 <span className="text
[activeCell.r + prevL][activeCell.c + prevL]
 </div>
 </>
)
 }}()
 </div>

 <div className="pt-3 mt-3 border-t
) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>
) : {
 <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L}
 </div>
)}
 </div>
)}
 </div>
 </div>
 </div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);

 const H = r2 - r1 + 1;

```



```

.....
 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: 'calc(' + (H/8*100) +
 width: 'calc(' + (W/8*100) +
 }}
 />

 /* Показываем 4 угла в самой матрице
 <div className="absolute pointer-events-none shadow-[0_0_10px_#f43f5e]" style={{ top: `calc(
 <div className="absolute pointer-events-none shadow-[0_0_10px_#3b82f6]" style={{ top: `calc(
 width: `calc(
 <div className="absolute pointer-events-none shadow-[0_0_10px_#10b981]" style={{ top: `calc(
 8px)`, width: `calc(
 <div className="absolute pointer-events-none shadow-[0_0_10px_#f59e0b]" style={{ top: `calc(
 00}% + 8px)`, width: `calc(
 </div>

 <div className="bg-slate-950 p-4 rounded-[10px] border border-slate-800 shadow-[inset_0_2px_10px_rgba(0,0,0,0.5)]" style={{
 <label className="flex items-center justify-between" style={{ padding: 0 10px }}>
 Матрица
 <div className="flex gap-2">
 <input type="number" min=0 max=100 value=50 />
 <div style="border: 1px solid #ccc; padding: 2px 5px; font-size: 0.8em; background-color: #f9f9f9;">
 14 bg-slate-900 border border-slate-700 py-1 px-2
 <input type="number" min=0 max=100 value=50 />
 <div style="border: 1px solid #ccc; padding: 2px 5px; font-size: 0.8em; background-color: #f9f9f9;">
 14 bg-slate-900 border border-slate-700 py-1 px-2
 </div>
 </div>
 </label>
 <label className="flex items-center justify-between" style={{ padding: 0 10px }}>
 Матрица
 <div className="flex gap-2">
 <input type="number" min=0 max=100 value=50 />

```



```
.....
 + 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
 </div>
 </div>
</div>
}
}
```

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

ФАЙЛ 53/61: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

```
.....
 </div>
 </main>
</div>

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal')...
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold'

 document.getElementById('btn-mode-wtf').cla...
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold'

 const aside = document.querySelector('aside')
 const headerMain = document.querySelector('h1')
 const questionsContainer = document.getElementByI...
 const wtfContainer = document.getElementById('wtf')
```

```

.....
 document.getElementById('btn-theme-light').
 ext-white transition-colors' : 'px-3 py-1 r
 ion-colors';
 document.getElementById('btn-theme-notebook
 -600 text-white transition-colors' : 'px-3
 ransition-colors';
 }

 // Initialize theme and mode
 setTheme('dark');

 // Setup intersection observer for TOC highlight
 document.addEventListener('DOMContentLoaded', () {
 const mobileMenuBtn = document.getElementById('mobile-menu-btn');
 const mobileDrawer = document.getElementById('mobile-drawer');
 const closeDrawerBtn = document.getElementById('close-drawer');
 const drawerOverlay = document.getElementById('drawer-overlay');

 function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains('closed');
 if (isClosed) {
 mobileDrawer.classList.remove("transition-colors");
 if (drawerOverlay) {
 drawerOverlay.classList.remove("opacity-0");
 // Timeout allows display:block
 setTimeout(() => drawerOverlay.classList.add("display-block"), 10);
 }
 document.body.style.overflow = "hidden";
 } else {
 mobileDrawer.classList.add("transition-colors");
 if (drawerOverlay) {
 drawerOverlay.classList.add("opacity-100");
 setTimeout(() => drawerOverlay.classList.remove("display-block"), 10);
 }
 document.body.style.overflow = "";
 }
 }
 });

```

```
.....
 });
 }, observerOptions);

 document.querySelectorAll('section[id^="que
 observer.observe(section);
 });
});
</script>
</body>
</html>
```

.....  
ФАЙЛ 54/61: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОВОК: 1169 | РАЗМЕР: 10.5 КБ

```
.....
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, ...>
 <title>Пособие по алгебре: От пиццы к предикатам</title>
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js"></script>
 <script id="MathJax-script" async src="https://cdn.mathjax.org/mathjax/latest/MathJax.js?config=TeX-AMS-MML_HTMLorMML"></script>
 <script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']],
 displayMath: [['$$', '$$'], ['\\[', '\\]']],
 processEscapes: true,
 },
 };
 </script>
 <style>
 @import url('https://fonts.googleapis.com/css2?family=Fira+Code&display=block');
 @reference './src/index.css';
 .math-font { font-family: 'Fira Code', monospace; }
```

```

.....
 -webkit-overflow-scrolling: touch;
}
mjsx-container[display="true"]::-webkit-scrollbar {
 display: none;
}

@media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font-size: 12px; }
 .uno-card::before, .uno-card::after { font-size: 12px; }
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px; }

 .uno-card.mini { width: 28px !important; height: 28px !important; font-size: 4px; }
 .uno-card.mini::before, .uno-card.mini::after { font-size: 4px; }

 .uno-equation .text-2xl { font-size: 1.2rem; }
 .uno-equation.gap-4 { gap: 0.5rem !important; }

 /* Allow horizontal scroll for equations in mobile view
 .formula-scroll { flex-wrap: nowrap !important; }
 .formula-scroll { width: 100%; }
}

.uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
}

.uno-val { position: relative; z-index: 2; }
.card-red { background-color: #e52521; } .card-blue { background-color: #004dcf; } .card-green { background-color: #339e35; } .card-yellow { background-color: #ffc400; } .card-black { background-color: #222; } .card-black::before, .card-black::after { text-align: center; }
.info-block { @apply bg-slate-800/80 border-l-4

```

```

}
body.theme-notebook .text-slate-200, body.theme
 ant; }
body.theme-notebook .text-slate-400 { color: #5
body.theme-notebook .border-slate-600, body.the
 : #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "
body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1);
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
}
.creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
}
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead
 <div class="sticky top-0 z-50 w-full bg-slate-950/9
 center items-center gap-4 sm:gap-8 transition-col
 <div class="flex items-center gap-3">

```

```

</marker>
<marker id="arrow-lime" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d" stroke="black" stroke-width="1" />
</marker>
<marker id="arrow-green" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80" stroke="black" stroke-width="1" />
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20" fill="#fde725" stroke="black" stroke-width="1" />
 <rect x="6" y="-5" width="8" height="6" fill="black" stroke="black" stroke-width="1" />
 <rect x="26" y="-5" width="8" height="6" fill="black" stroke="black" stroke-width="1" />
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
</g>
<g id="lego-red">
 <rect x="0" y="0" width="40" height="20" fill="#e34a33" stroke="black" stroke-width="1" />
 <rect x="6" y="-5" width="8" height="6" fill="black" stroke="black" stroke-width="1" />
 <rect x="26" y="-5" width="8" height="6" fill="black" stroke="black" stroke-width="1" />
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
</g>
<g id="lego-rem">
 <rect x="0" y="0" width="40" height="10" fill="white" stroke="black" stroke-width="1" />
 <rect x="6" y="-5" width="8" height="6" fill="black" stroke="black" stroke-width="1" />
 <rect x="26" y="-5" width="8" height="6" fill="black" stroke="black" stroke-width="1" />
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
</g>
<!-- МЯСОРУБКА -->
<g id="meat-grinder">
 <path d="M 10 40 L 15 20 Q 25 15 35 20 L 40 20" fill="white" stroke="black" stroke-width="1" />
 <rect x="5" y="40" width="40" height="20" fill="white" stroke="black" stroke-width="1" />
 <!-- Ручка -->
 <path d="M 45 50 Q 60 50 60 30 L 75 30" fill="white" stroke="black" stroke-width="1" />
 <circle cx="75" cy="30" r="4" fill="#333" stroke="black" stroke-width="1" />
 <!-- Выход -->
 <path d="M 5 45 L -5 45 L -5 55 L 5 55" fill="white" stroke="black" stroke-width="1" />
 <circle cx="25" cy="50" r="15" fill="white" stroke="black" stroke-width="1" />
</g>

```



```
.....
 <button id="close-drawer-btn" class="
 <svg xmlns="http://www.w3.org/20
 <path stroke-linecap="round
 </svg>
 </button>
 </div>
 <nav class="space-y-3 text-sm font-medium"
 <details class="group" open>
 <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-300">
 1. Алгебраические структуры

 </summary>
 <div class="pl-6 space-y-2 text-sm">

 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-indigo-300">
 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text-sm">

 </div>
 </details>

 <details class="group">
```

```
.....

</div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-lime-500 pl-3 font-bold text-lime-300">

 6. Делимость, НОД и Евклид

 </summary>
 <div class="pl-6 space-y-2 text-lime-300">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-pink-500 pl-3 font-bold text-pink-300">

 7. Взаимно простые числа

 </summary>
 <div class="pl-6 space-y-2 text-pink-300">


```

```
.....
 <a href="#question-10" class=
der-rose-500 pl-3 font-bold text-rose-400">
 10. Делимость и Схема Г

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q10-1" class="blo
 <a href="#q10-2" class="blo
 <a href="#q10-3" class="blo
a>
 <a href="#q10-4" class="blo
 <a href="#q10-5" class="blo
 </div>
</details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-11" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
 11. НОД и НОК многочлен

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q11-1" class="blo
 <a href="#q11-2" class="blo
нственности НОД
 <a href="#q11-3" class="blo
ПИРАЛЬ)
 <a href="#q11-4" class="blo
вращается!)
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-12" clas
der-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые мно
```

```
.....
 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">

 15. Производная. Критерий

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">

 16. Неприводимые многочлены

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <div class="mt-4 mb-2 text-xs font-sans text-slate-500 pl-3 text-slate-400">


```

```

// Scroll spy logic
document.addEventListener('DOMContentLoaded', () => {
 const sections = document.querySelectorAll(
 "#q5", div[id^="q6"], div[id^="q7"], div[id^="q8"],
 div[id^="q9"], div[id^="q10"], div[id^="q11"], div[id^="q12"],
 div[id^="q13"], div[id^="q14"], div[id^="q15"]
);
 let isScrolling = false;

 document.querySelectorAll('.grow').forEach(link => {
 link.addEventListener('click', () => {
 isScrolling = true;
 setTimeout(() => isScrolling = false, 200);
 });
 });

 const observer = new IntersectionObserver(entries => {
 if (isScrolling) return;

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 visibleId = entry.target.id;
 }
 });

 // Highlight active link
 document.querySelectorAll('a').forEach(a => {
 a.style.fontWeight = 'normal';
 if (a.href.endsWith(visibleId)) {
 a.style.fontWeight = 'bold';
 }
 });

 // Open the parent details
 const correspondingLi = document.querySelector(`li[id^="${visibleId}"]`);
 if (correspondingLi) {
 const details = correspondingLi.querySelector('details');
 if (details && !details.open) {
 details.open();
 }
 }
 });
});

```

```
.....
 box.style.borderRadius = '10px'
 box.style.display = 'flex';
 box.style.flexDirection = 'column'
 box.style.color = '#fff';

 const header = document.createElement('div')
 header.innerText = 'Экспорт в '
 header.style.marginBottom = '10px'

 const subHeader = document.createElement('div')
 subHeader.innerText = 'Из-за проблем с
ние веб-студии в новой вкладке (через меню)
 subHeader.style.marginBottom = '10px'
 subHeader.style.color = '#f87171'
 subHeader.style.fontSize = '0.9em'

 const textarea = document.createElement('div')
 textarea.value = contentRaw;
 textarea.style.flex = '1';
 textarea.style.width = '100%';
 textarea.style.color = '#000';
 textarea.style.padding = '10px'
 textarea.style.fontFamily = 'monospace'

 const btnContainer = document.createElement('div')
 btnContainer.style.display = 'flex'
 btnContainer.style.gap = '10px'
 btnContainer.style.marginTop = '10px'

 const copyBtn = document.createElement('div')
 copyBtn.innerText = 'Копировать'
 copyBtn.style.padding = '10px';
 copyBtn.style.backgroundColor = 'white'
 copyBtn.style.color = '#fff';
 copyBtn.style.border = 'none';
 copyBtn.style.borderRadius = '5px'
 copyBtn.style.cursor = 'pointer'
 copyBtn.onclick = () => {
```

```

.....
// Smart markdown parsing based
clone.querySelectorAll('h2').fo
clone.querySelectorAll('h3').fo
clone.querySelectorAll('.analog
clone.querySelectorAll('li').fo
clone.querySelectorAll('svg').fo
 let caption = '';
 if (el.nextElementSibling &
 caption = el.nextElement
 }
 el.textContent = '\n\n{ Илл
});
clone.querySelectorAll('.img-ca

let text = clone.innerText;
text = text.replace(/\\n{3,}/g,

if (window !== window.top) {
 fallbackModal(text, 'Markdow
 return;
}
const blob = new Blob([text], {
const url = URL.createObjectURL
const link = document.createEle
link.href = url;
link.download = 'vyisshaya_alge
document.body.appendChild(link)
link.click();
document.body.removeChild(link)
}

function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate

);

document.getElementById('theme-

```

```

body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside {
#334155 !important; }
body.bg-white .text-white

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuch
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind

`
} else if (theme === 'terminal')
body.classList.add('bg-black')
document.getElementById('th
style.innerHTML = `
body.bg-black .bg-slate
.bg-slate-950 {

```



```
.....
 <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center
 HTML
 </button>
 </div>

 <!-- Смена темы -->
 <div>
 <div class="text-xs text-slate-500">
 <div class="flex items-center gap-2">
 <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
 <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white
">* </button>
 <button id="theme-terminal"
-center justify-center text-green-400 hover:text-green-500
терминала"> </button>
 </div>
 </div>
 </div>

</div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
-base min-w-0">
 <header class="text-center mb-12 border-b border-slate-700">
 <h1 class="text-4xl md:text-5xl font-bold text-slate-100">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg">
 Универсальное пособие
 </p>
 </header>

 <!-- Print Table of Contents -->
 <div class="hidden print:block mt-8 text-slate-100">
```

```

.....
 ольцо полиномов</div>
 <div class="bg-cyan-900/40 border-4 border-yellow-500 hover:bg-slate-700 transition" style="border: 4px solid yellow; background-color: cyan; color: yellow; padding: 5px; text-align: center;">
 сные числа</div>

 <!-- Row 1: База -->

 <div class="text-xs text-yellow" style="font-size: 0.8em; margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="font-weight: bold; margin: 0;">
 База
 </div>
 </div>

 <div class="text-xs text-orange" style="font-size: 0.8em; margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="font-weight: bold; margin: 0;">
 Базовые операции / Евклид
 </div>
 </div>

 <div class="text-xs text-cyan" style="font-size: 0.8em; margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="font-weight: bold; margin: 0;">
 Базовые операции / Евклид
 </div>
 </div>

 <div class="text-xs text-yellow" style="font-size: 0.8em; margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="font-weight: bold; margin: 0;">
 Базовые операции / Евклид
 </div>
 </div>

 <div class="text-xs text-orange" style="font-size: 0.8em; margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="font-weight: bold; margin: 0;">
 Базовые операции / Евклид
 </div>
 </div>

 <div class="text-xs text-cyan" style="font-size: 0.8em; margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="font-weight: bold; margin: 0;">
 Базовые операции / Евклид
 </div>
 </div>

 </div>
 </div>
 </div>
 </div>
 </div>

```

```

.....
 <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>
 <a href="#question-14" onclick="setf
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <div class="hidden md:block"></div>

 <!-- Row 7: Производная -->
 <div class="hidden md:block"></div>
 <a href="#question-15" onclick="setf
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <div class="hidden md:block"></div>

 <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
 <div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

 <a href="#question-17" onclick="setf
-1-4 border-teal-500 hover:bg-slate-700 tra
 <div class="text-xs text-teal-5
 <div class="font-bold text-slate

 <a href="#question-18" onclick="setf
-1-4 border-blue-500 hover:bg-slate-700 tra
 <div class="text-xs text-blue-5
 <div class="font-bold text-slate


```

```
.....
 <svg class="w-8 h-8 text-red-500" data-bbox="110 40 150 60"
rg/2000/svg">
 <path stroke-linecap="round" data-bbox="110 40 150 60"
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4" data-bbox="110 40 150 60"
 </svg>
 </div>
 <h2 class="text-3xl pr-4 border-l-2 border-l-blue-500" data-bbox="110 100 150 120">
</div>

<div class="prose prose-invert max-w-4xl" data-bbox="110 140 150 160">
 <p class="text-lg" data-bbox="110 140 150 160">
 Хватит тупить. Доказательств
 </p>
 <p data-bbox="110 160 150 180">есть для того, чтобы жать на кнопки. Вот жес
 </p>

 <h2 class="text-2xl font-bold border-l-2 border-l-blue-500" data-bbox="110 180 150 200">
(Меметика)</h2>

 <!-- Прямое -->
 <div class="bg-slate-800/80 p-6" data-bbox="110 220 150 240">
 <h3 class="text-xl text-blue-400" data-bbox="110 220 150 240">
 <p class="text-sm text-slate-400" data-bbox="110 240 150 260">
 <ul class="list-disc pl-5" data-bbox="110 260 150 280">
 <span class="text-w
k$).
 <span class="text-w
 <span class="text-w
ово.

 </div>

 <!-- От противного -->
 <div class="bg-slate-800/80 p-6" data-bbox="110 300 150 320">
 <h3 class="text-xl text-fuc
 <p class="text-sm text-slate
 <ul class="list-disc pl-5" data-bbox="110 320 150 340">
 <span class="text-w
i>
```

```
.....
 <ul class="list-disc pl-5 s
 <span class="text-w
вверх по уравнениям).
 <span class="text-w
рху вниз, показывая, что Δr_i
 <span class="text-w
х. Твой d больше.

 </div>
```

```

 <h2 class="text-2xl font-bold b
оритмы экзекуции)</h2>
```

```

 <div class="grid grid-cols-1 md
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
со следующим. Если в конце 0 – пациент мерт
 </div>
```

```

 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
 c . Пока получается 0 – корень жив. Как то
корня минус один.</p>
 </div>
```

```

 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
ициентов (они делятся на p), кроме старше
зложить.</p>
 </div>
```

```

.....
 <h4 class="text-cyan">
 <p class="text-sm">
 есть сложение — есть ноль. Если есть умноже
 <code class="text-x">
 </div>

 <li class="bg-slate-800/50">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 авь их драться. Умножение всегда вьрывается
 <code class="text-x">
 </div>

</div>
</section>
</div>

<div id="questions-container">
<!-- ... ВОПРОС 1 ...

```

РАЗДЕЛ: УТИЛИТЫ

ФАЙЛ 55/61: src/utlils/cn.ts  
 КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 B

```

import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

```

```

.....
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

const app = fs.readFileSync(path.join(ROOT, "src/App.ts"));
const vizIds = new Set([...app.matchAll(/activeChapter\w+/g)].map(m => m[1]));

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length;
 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\\s>]/i.test(html),
 image: /<img[\\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 };
});

const has2D = (r) => r.interactive || r.inlineSvg || r.image;
const gapBoth = rows.filter((r) => !r.hasAnalogy && !has2D(r));
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && has2D(r));
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D(r));
const orphanViz = [...vizIds].filter((id) => !chapters[id]);

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) => i + 1).filter(n => !nums.includes(n));

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG | Изображение |");
 console.log("| --- | --- | --- | --- | --- | --- |");
}

```

Универсальное пособие • полный исходный код

```
.....
const A4 = { width: 595.28, height: 841.89 };
```

```
async function main() {
 if (!fs.existsSync(PAGES_DIR)) {
 throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
 }
```

```
 const pages = fs
 .readdirSync(PAGES_DIR)
 .filter((f) => f.endsWith(".png"))
 .sort();
```

```
 if (pages.length === 0) throw new Error("Не найдено ни одной страницы");
```

```
 ensureDir(OUT_DIR);
```

```
 const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
 });
```

```
 const stream = fs.createWriteStream(PDF_PATH);
 doc.pipe(stream);
```

```
 for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width, height: A4.height });
 }
```

```
 doc.end();
 await new Promise((resolve, reject) => {
```



```
});
```

```
/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",
```

```
/** Человекочитаемые категории для оглавления. */
```

```
const CATEGORIES = [
 [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
 [/^src\/data\/$/, "Контент и данные пособия"],
 [/^src\/components\/visualizers\/$/, "Визуализаторы (в
 [/^src\/components\/$/, "Интерактивные компоненты и сим
 [/^src\/layout\/$/, "HTML-разметка (layout)"],
 [/^src\/utils\/$/, "Утилиты"],
 [/^src\/$/, "Ядро приложения"],
 [/^scripts\/$/, "Скрипты сборки и экспорта"],
 [/^\.github\/$/, "CI / автоматизация"],
 [/^[^\/]+$/, "Конфигурация проекта"],
];
```

```
const categoryOf = (rel) => CATEGORIES.find(({re}) => re.test(rel))
```

```
/** Порядок разделов в документе. */
```

```
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];
```

```
function listFiles() {
```

## Универсальное пособие • полный исходный код

```
.....
for (const rel of files.filter((f) => f.startsWith("s
 const src = fs.readFileSync(path.join(ROOT, rel), "
 const re = /id:\s*"([^\"]+)"\s*,\s*\n?\s*title:\s*"([
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 chapters.push({ id, title, file: rel });
 }
}
return chapters.sort((a, b) => {
 const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
 const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
 return na - nb;
}));
}

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { ti
 const totalBytes = files.reduce((s, f) => s + fs.stat

 push(HR);
 push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТ
 push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФА
 push(HR);
 push();
 push(`Дата генерации:                ${stamp}`);
 push(`Файлов исходного кода:         ${files.length}`);
 push(`Суммарный объём кода: ${humanSize(totalByt
 push(`Глав/билетов в пособии:         ${chapters.length}`);
```

```

.....
 push(SUBHR);
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} | `);
 push(SUBHR);
 push();
 push(content.replace(/\n+$/, ""));
 push();
 push();
 });

 push(HR);
 push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
 push(HR);

 const txt = out.join("\n") + "\n";
 fs.writeFileSync(TXT_PATH, txt, "utf8");

 console.log("[1/3] код -> txt");
 console.log(`      файлов:  ${files.length}`);
 console.log(`      глав:    ${chapters.length}`);
 console.log(`      строк:   ${txt.split("\n").length}`);
 console.log(`      выход:   ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```

ФАЙЛ 59/61: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМЕР: 1.1 КБ

```

/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt (collect-code.mjs)
 * txt -> page-XXXX.png (render-pages.mjs)
 * png -> full_code_all_pages.pdf (build-pdf.mjs)

```

```
.....

export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
 for (const cmd of ["magick", "convert"]) {
 try {
 execFileSync(cmd, ["-version"], { stdio: "ignore" });
 return cmd;
 } catch {
 /* пробуем следующий */
 }
 }
 throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-get install imagemagick"
);
}
```

.....  
ФАЙЛ 60/61: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 1.1 KB

```
.....

/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
```

```

}

const im = imageMagickCmd();
const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

fs.rmSync(PAGES_DIR, { recursive: true, force: true });
ensureDir(PAGES_DIR);

const total = pages.length;
const jobs = pages.map(({lines, idx}) => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`)
 `стр. ${idx + 1} / ${total}`;
 const body = [header, "-".repeat(PAGE.cols), ...lines].join("\n");
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
});

const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "DejaVuSansMono",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;

```

Универсальное пособие • полный исходный код

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 128 | РАЗМЕР: 4.1

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

#  
# исходный код → full\_code\_all\_pages.txt (script)  
#  
#       └─→ page-0001.png ... page-NNNN.png (artifacts)  
#  
#                   └─→ full\_code\_all\_page.pdf (artifact)

# Готовые TXT и PDF коммитятся обратно в ветку (их отдаёт GitHub)  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бесконечно

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

density:

description: "DPI растеризации страниц (по умолчанию 150)"

required: false

default: "150"

concurrency:

group: rebuild-pdf-\${{ github.ref }}

cancel-in-progress: true

permissions:

contents: write

.....

- name: Step 2 – txt → png  
env:  
  EXPORT\_DENSITY: \${ github.event.inputs.density }  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
  {  
    echo "### Экспорт пересобран"  
    echo ""  
    echo "| Артефакт | Размер |"  
    echo "| --- | --- |"  
    echo "| \`public/export/full\_code\_all\_pages`"  
    echo "| \`public/export/full\_code\_all\_pages`"  
    echo "| PNG-страниц | \$(ls tmp/export-pages)"  
  } >> "\$GITHUB\_STEP\_SUMMARY"
- name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v4  
with:  
  name: full-code-export  
  path: |  
    public/export/full\_code\_all\_pages.pdf  
    public/export/full\_code\_all\_pages.txt  
  if-no-files-found: error  
  retention-days: 30
- name: Upload artifacts (PNG-страницы)  
uses: actions/upload-artifact@v4  
with: